

4.0 System Design

This chapter describes the activities in the system design processes listed in Figure 2.1-1. The chapter is separated into sections corresponding to steps 1 to 4 listed in Figure 2.1-1. The processes within each step are discussed in terms of inputs, activities, and outputs. Additional guidance is provided using examples that are relevant to NASA projects. The system design processes are four interdependent, highly iterative and recursive processes, resulting in a validated set of requirements and a validated design solution that satisfies a set of stakeholder expectations. The four system design processes are to develop stakeholder expectations, technical requirements, logical decompositions, and design solutions.

Figure 4.0-1 illustrates the recursive relationship among the four system design processes. These processes start

with a study team collecting and clarifying the stakeholder expectations, including the mission objectives, constraints, design drivers, operational objectives, and criteria for defining mission success. This set of stakeholder expectations and high-level requirements is used to drive an iterative design loop where a strawman architecture/design, the concept of operations, and derived requirements are developed. These three products must be consistent with each other and will require iterations and design decisions to achieve this consistency. Once consistency is achieved, analyses allow the project team to validate the design against the stakeholder expectations. A simplified validation asks the questions: Does the system work? Is the system safe and reliable? Is the system achievable within budget and schedule constraints? If the answer to any of these questions is no,

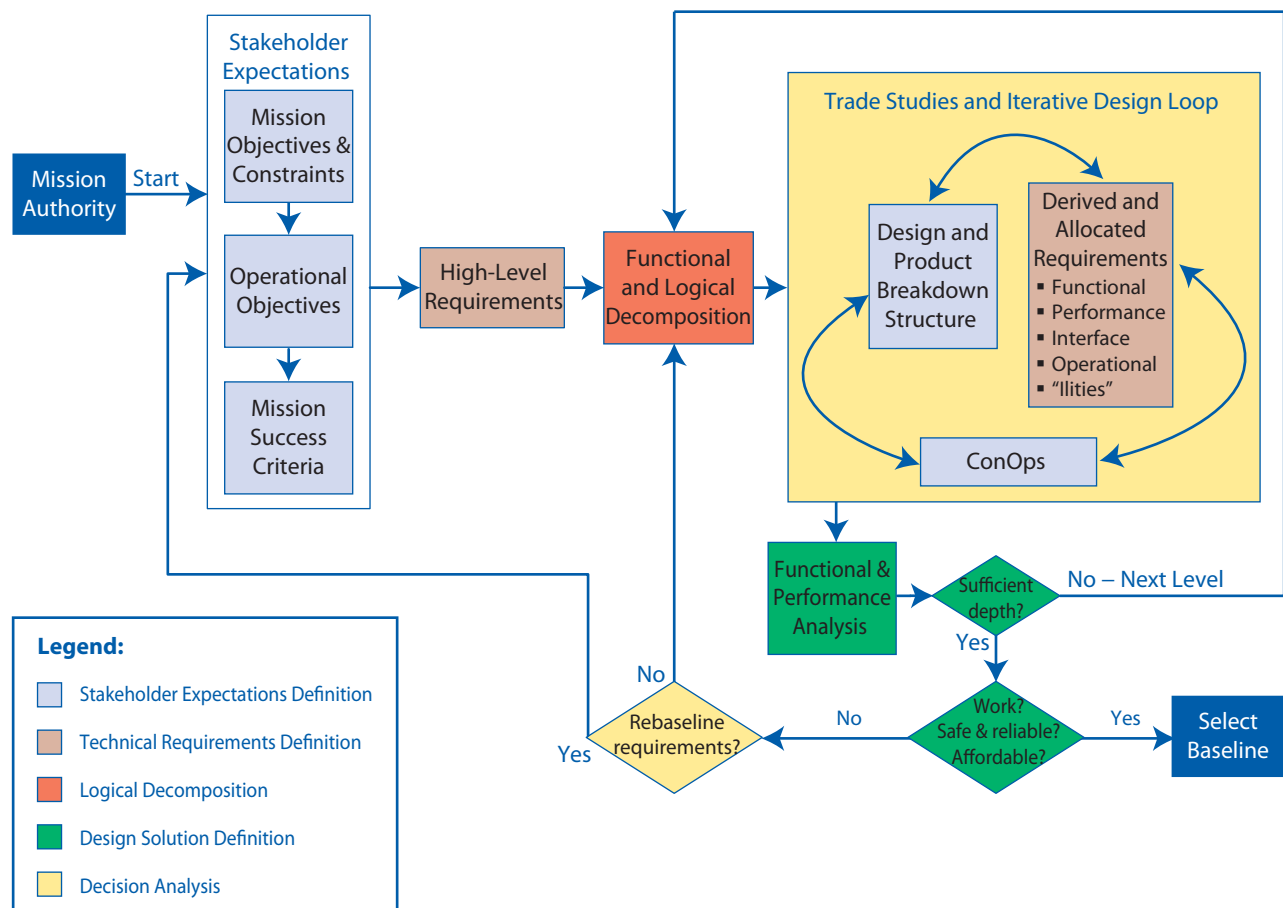


Figure 4.0-1 Interrelationships among the system design processes

4.0 System Design

then changes to the design or stakeholder expectations will be required, and the process started again. This process continues until the system—architecture, ConOps, and requirements—meets the stakeholder expectations.

The depth of the design effort must be sufficient to allow analytical verification of the design to the requirements. The design must be feasible and credible when judged by a knowledgeable independent review team and must have sufficient depth to support cost modeling.

Once the system meets the stakeholder expectations, the study team baselines the products and prepares for the next phase. Often, intermediate levels of decomposition are validated as part of the process. In the next level of decomposition, the baselined derived (and allocated) requirements become the set of high-level requirements for the decomposed elements and the process begins again. These system design processes are primarily applied in Pre-Phase A and continue through Phase C.

The system design processes during Pre-Phase A focus on producing a feasible design that will lead to Formulation approval. During Phase A, alternative designs and additional analytical maturity are pursued to optimize the design architecture. Phase B results in a preliminary design that satisfies the approval criteria. During Phase C, detailed, build-to designs are completed.

This has been a simplified description intended to demonstrate the recursive relationship among the system design processes. These processes should be used as guidance and tailored for each study team depending on the size of the project and the hierarchical level of the study

team. The next sections describe each of the four system design processes and their associated products for a given NASA mission.

System Design Keys

- Successfully understanding and defining the mission objectives and operational concepts are keys to capturing the stakeholder expectations, which will translate into quality requirements over the life cycle of the project.
- Complete and thorough requirements traceability is a critical factor in successful validation of requirements.
- Clear and unambiguous requirements will help avoid misunderstanding when developing the overall system and when making major or minor changes.
- Document all decisions made during the development of the original design concept in the technical data package. This will make the original design philosophy and negotiation results available to assess future proposed changes and modifications against.
- The design solution verification occurs when an acceptable design solution has been selected and documented in a technical data package. The design solution is verified against the system requirements and constraints. However, the validation of a design solution is a continuing recursive and iterative process during which the design solution is evaluated against stakeholder expectations.

4.1 Stakeholder Expectations Definition

The Stakeholder Expectations Definition Process is the initial process within the SE engine that establishes the foundation from which the system is designed and the product is realized. The main purpose of this process is to identify who the stakeholders are and how they intend to use the product. This is usually accomplished through use-case scenarios, Design Reference Missions (DRMs), and ConOps.

4.1.1 Process Description

Figure 4.1-1 provides a typical flow diagram for the Stakeholder Expectations Definition Process and identifies typical inputs, outputs, and activities to consider in addressing stakeholder expectations definition.

4.1.1.1 Inputs

Typical inputs needed for the Stakeholder Expectations Definition Process would include the following:

- **Upper Level Requirements and Expectations:** These would be the requirements and expectations (e.g., needs, wants, desires, capabilities, constraints, external interfaces) that are being flowed down to a particular system of interest from a higher level (e.g., program, project, etc.).
- **Identified Customers and Stakeholders:** The organization or individual who has requested the product(s) and those who are affected by or are in some way accountable for the product's outcome.

4.1.1.2 Process Activities

Identifying Stakeholders

Advocacy for new programs and projects may originate in many organizations. These include Presidential directives, Congress, NASA Headquarters (HQ), the NASA Centers, NASA advisory committees, the National Academy of Sci-

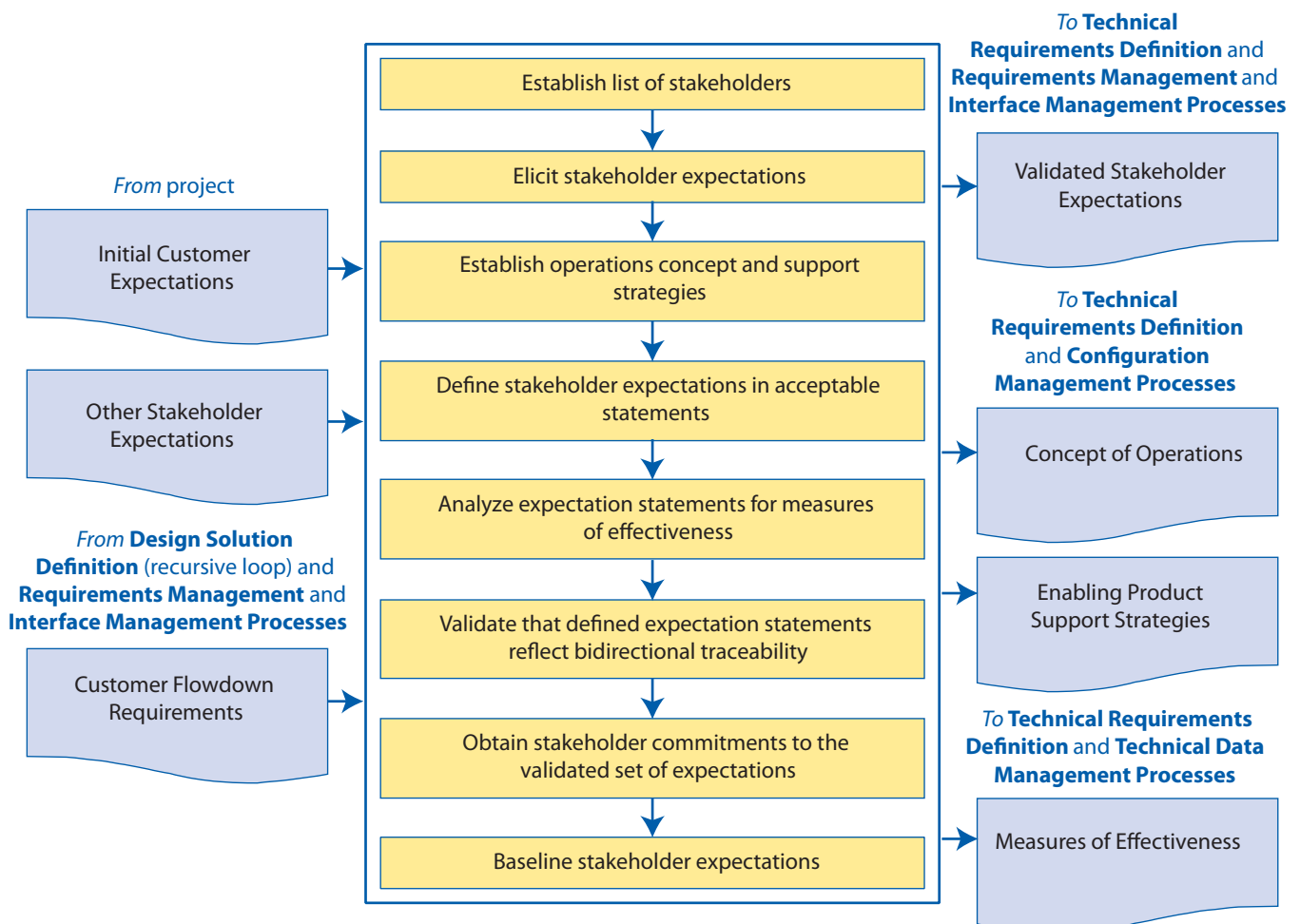


Figure 4.1-1 Stakeholder Expectations Definition Process

4.0 System Design

ences, the National Space Council, and many other groups in the science and space communities. These organizations are commonly referred to as stakeholders. A stakeholder is a group or individual who is affected by or is in some way accountable for the outcome of an undertaking.

Stakeholders can be classified as customers and other interested parties. Customers are those who will receive the goods or services and are the direct beneficiaries of the work. Examples of customers are scientists, project managers, and subsystems engineers.

Other interested parties are those who affect the project by providing broad, overarching constraints within which the customers' needs must be achieved. These parties may be affected by the resulting product, the manner in which the product is used, or have a responsibility for providing life-cycle support services. Examples include Congress, advisory planning teams, program managers, users, operators, maintainers, mission partners, and NASA contractors. It is important that the list of stakeholders be identified early in the process, as well as the primary stakeholders who will have the most significant influence over the project.

Identifying Stakeholder Expectations

Stakeholder expectations, the vision of a particular stakeholder individual or group, result when they specify what is desired as an end state or as an item to be produced and put bounds upon the achievement of the goals. These bounds may encompass expenditures (resources), time to deliver,

performance objectives, or other less obvious quantities such as organizational needs or geopolitical goals.

Figure 4.1-2 shows the type of information needed when defining stakeholder expectations and depicts how the information evolves into a set of high-level requirements. The yellow paths depict validation paths. Examples of the types of information that would be defined during each step are also provided.

Defining stakeholder expectations begins with the mission authority and strategic objectives that the mission is meant to achieve. *Mission authority* changes depending on the category of the mission. For example, science missions are usually driven by NASA Science Mission Directorate strategic plans; whereas the exploration missions may be driven by a Presidential directive.

An early task in defining stakeholder expectations is understanding the *objectives of the mission*. Clearly describing and documenting them helps ensure that the project team is working toward a common goal. These objectives form the basis for developing the mission, so they need to be clearly defined and articulated.

Defining the objectives is done by eliciting the needs, wants, desires, capabilities, external interfaces, assumptions, and constraints from the stakeholders. Arriving at an agreed-to set of objectives can be a long and arduous task. The proactive iteration with the stakeholders throughout the systems engineering process is the way

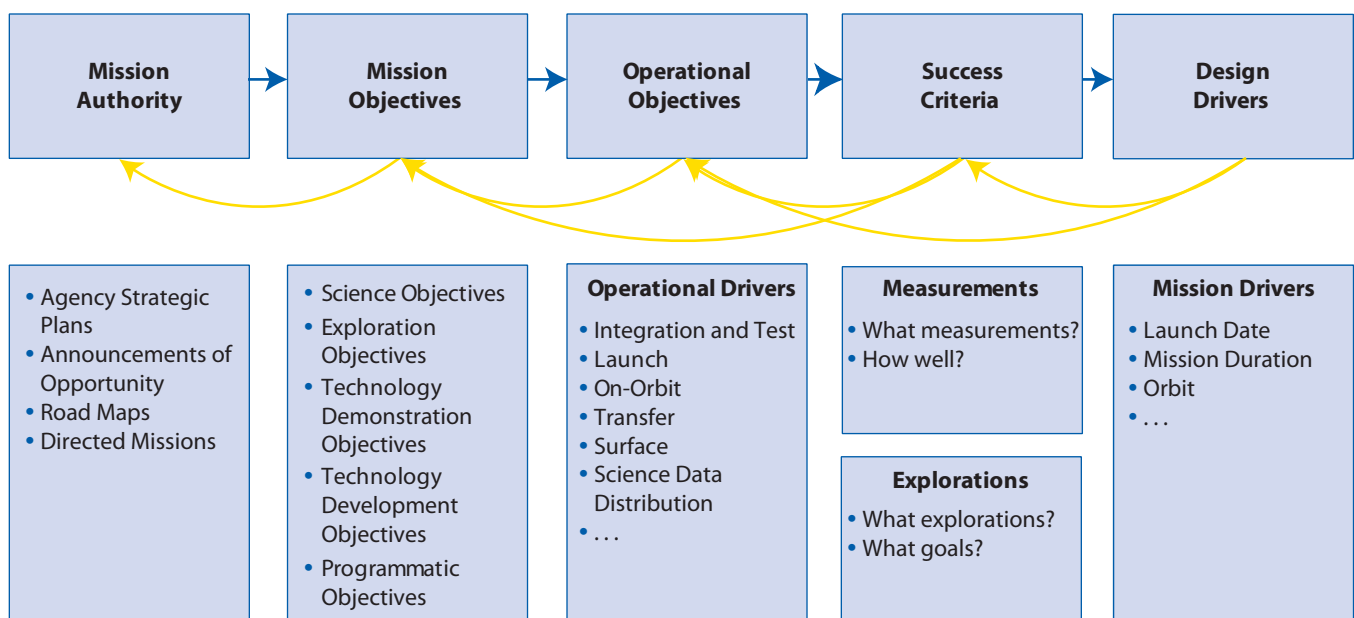


Figure 4.1-2 Product flow for stakeholder expectations

that all parties can come to a true understanding of what should be done and what it takes to do the job. It is important to know who the primary stakeholders are and who has the decision authority to help resolve conflicts.

The project team should also identify the *constraints* that may apply. A constraint is a condition that must be met. Sometimes a constraint is dictated by external factors such as orbital mechanics or the state of technology; sometimes constraints are the result of the overall budget environment. It is important to document the constraints and assumptions along with the mission objectives.

Operational objectives also need to be included in defining the stakeholder expectations. The operational objectives identify how the mission must be operated to achieve the mission objectives.

The *mission and operational success criteria* define what the mission must accomplish to be successful. This will be in the form of a measurement concept for science missions and exploration concept for human exploration missions. The success criteria also define how well the concept measurements or exploration activities must be accomplished. The success criteria capture the stakeholder expectations and, along with programmatic requirements and constraints, are used within the high-level requirements.

The *design drivers* will be strongly dependent upon the ConOps, including the operational environment, orbit, and mission duration requirements. For science missions, the design drivers may include, at a minimum, the mission launch date, duration, and orbit. If alternative orbits are to be considered, a separate concept is needed for each orbit. Exploration missions must consider the destination, the duration, the operational sequence (and system configuration changes), and the in situ exploration activities that allow the exploration to succeed.

The end result of this step is the discovery and delineation of the *system's goals*, which generally express the agree-

ments, desires, and requirements of the eventual users of the system. The high-level requirements and success criteria are examples of the products representing the consensus of the stakeholders.

4.1.1.3 Outputs

Typical outputs for capturing stakeholder expectations would include the following:

- **Top-Level Requirements and Expectations:** These would be the top-level requirements and expectations (e.g., needs, wants, desires, capabilities, constraints, and external interfaces) for the product(s) to be developed.
- **ConOps:** This describes how the system will be operated during the life-cycle phases to meet stakeholder expectations. It describes the system characteristics from an operational perspective and helps facilitate an understanding of the system goals. Examples would be the ConOps document or a DRM.

4.1.2 Stakeholder Expectations Definition Guidance

4.1.2.1 Concept of Operations

The ConOps is an important component in capturing stakeholder expectations, requirements, and the architecture of a project. It stimulates the development of the requirements and architecture related to the user elements of the system. It serves as the basis for subsequent definition documents such as the operations plan, launch and early orbit plan, and operations handbook and provides the foundation for the long-range operational planning activities such as operational facilities, staffing, and network scheduling.

The ConOps is an important driver in the system requirements and therefore must be considered early in the system design processes. Thinking through the ConOps and use cases often reveals requirements and design functions that might otherwise be overlooked. A simple example to illustrate this point is adding system requirements to allow for communication during a particular phase of a mission. This may require an additional antenna in a specific location that may not be required during the nominal mission.

The ConOps is important for all projects. For science projects, the ConOps describes how the systems will be operated to achieve the measurement set required for a

Note: It is extremely important to involve stakeholders in all phases of a project. Such involvement should be built in as a self-correcting feedback loop that will significantly enhance the chances of mission success. Involving stakeholders in a project builds confidence in the end product and serves as a validation and acceptance with the target audience.

4.0 System Design

successful mission. They are usually driven by the data volume of the measurement set. The ConOps for exploration projects is likely to be more complex. There are typically more operational phases, more configuration changes, and additional communication links required for human interaction. For human spaceflight, functions and objectives must be clearly allocated between human operators and systems early in the project.

The ConOps should consider all aspects of operations including integration, test, and launch through disposal. Typical information contained in the ConOps includes a description of the major phases; operation timelines; operational scenarios and/or DRM; end-to-end communications strategy; command and data architecture; operational facilities; integrated logistic support (resupply, maintenance, and assembly); and critical events. The operational scenarios describe the dynamic view of the systems' operations and include how the system is perceived to function throughout the various modes and mode transitions, including interactions with external inter-

faces. For exploration missions, multiple DRMs make up a ConOps. The design and performance analysis leading to the requirements must satisfy all of them. Figure 4.1-3

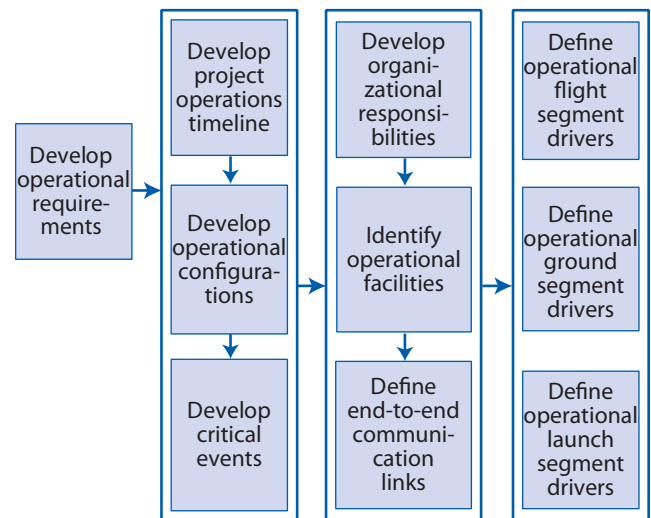


Figure 4.1-3 Typical ConOps development for a science mission

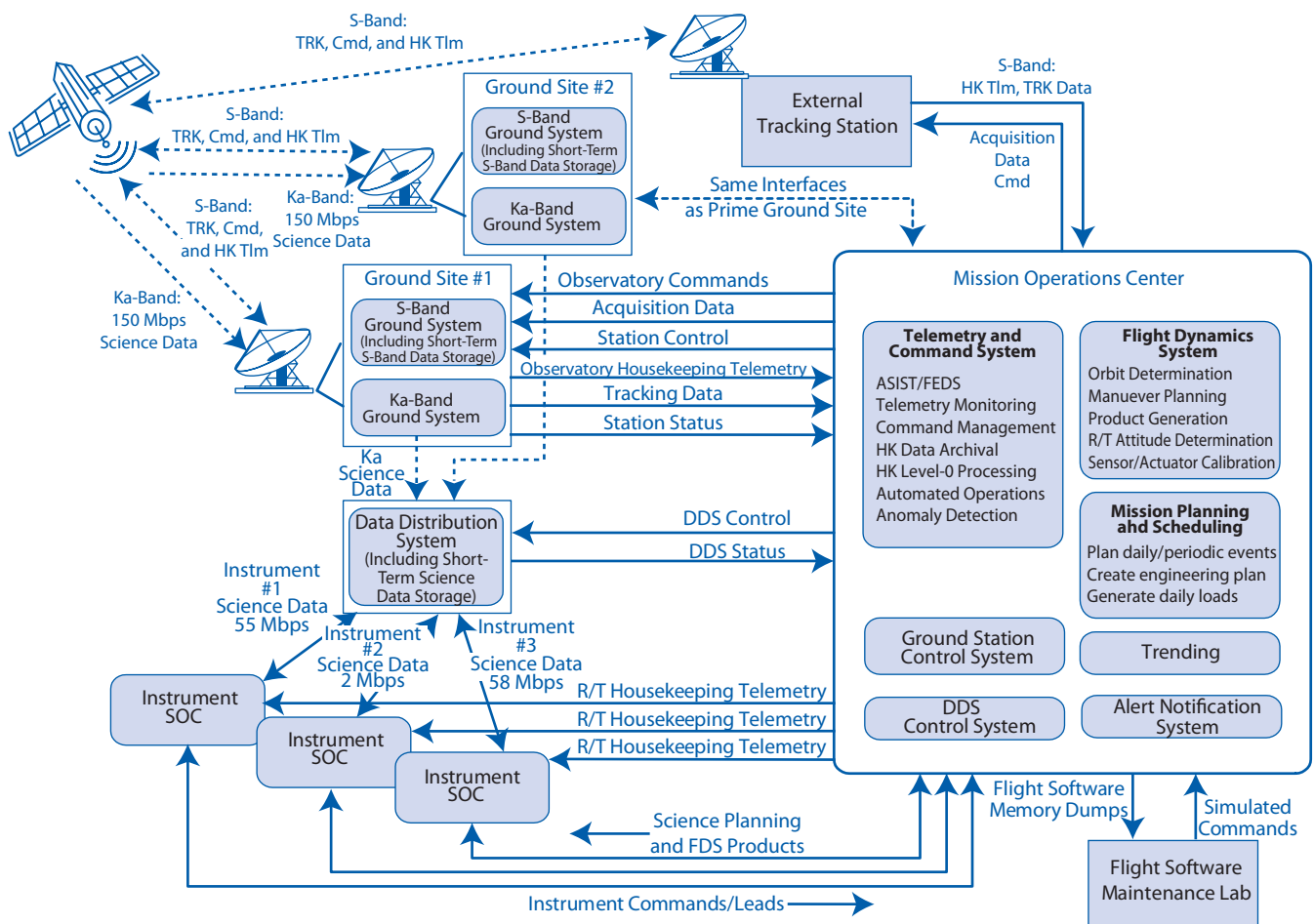


Figure 4.1-4 Example of an associated end-to-end operational architecture

illustrates typical information included in the ConOps for a science mission, and Figure 4.1-4 is an example of an end-to-end operational architecture. For more information about developing the ConOps, see ANSI/AIAA G-043-1992, *Guide for the Preparation of Operational Concept Documents*.

The operation timelines provide the basis for defining system configurations, operational activities, and other sequenced related elements necessary to achieve the mission objectives for each operational phase. It describes the activities, tasks, and other sequenced related elements necessary to achieve the mission objectives in each of the phases. Depending on the type of project (science, exploration, operational), the timeline could become quite complex.

The timeline matures along with the design. It starts as a simple time-sequenced order of the major events and matures into a detailed description of subsystem operations during all major mission modes or transitions. An example of a lunar sortie timeline and DRM early in the life cycle are shown in Figures 4.1-5a and b, respectively. An example of a more detailed, integrated time-

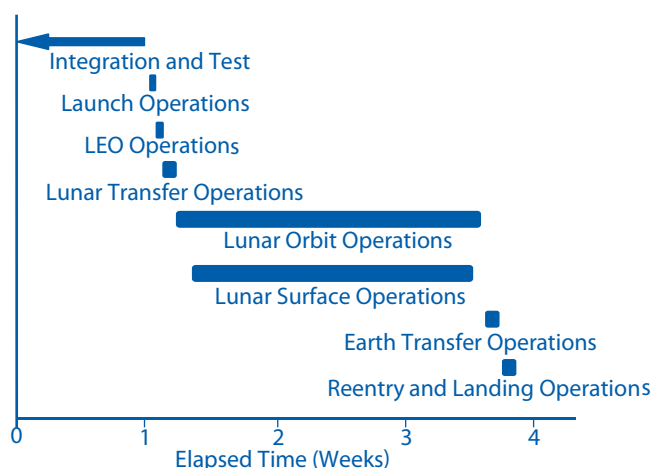


Figure 4.1-5a Example of a lunar sortie timeline developed early in the life cycle

line later in the life cycle for a science mission is shown in Figure 4.1-6.

An important part of the ConOps is defining the operational phases, which will span project Phases D, E, and F. The operational phases provide a time-sequenced

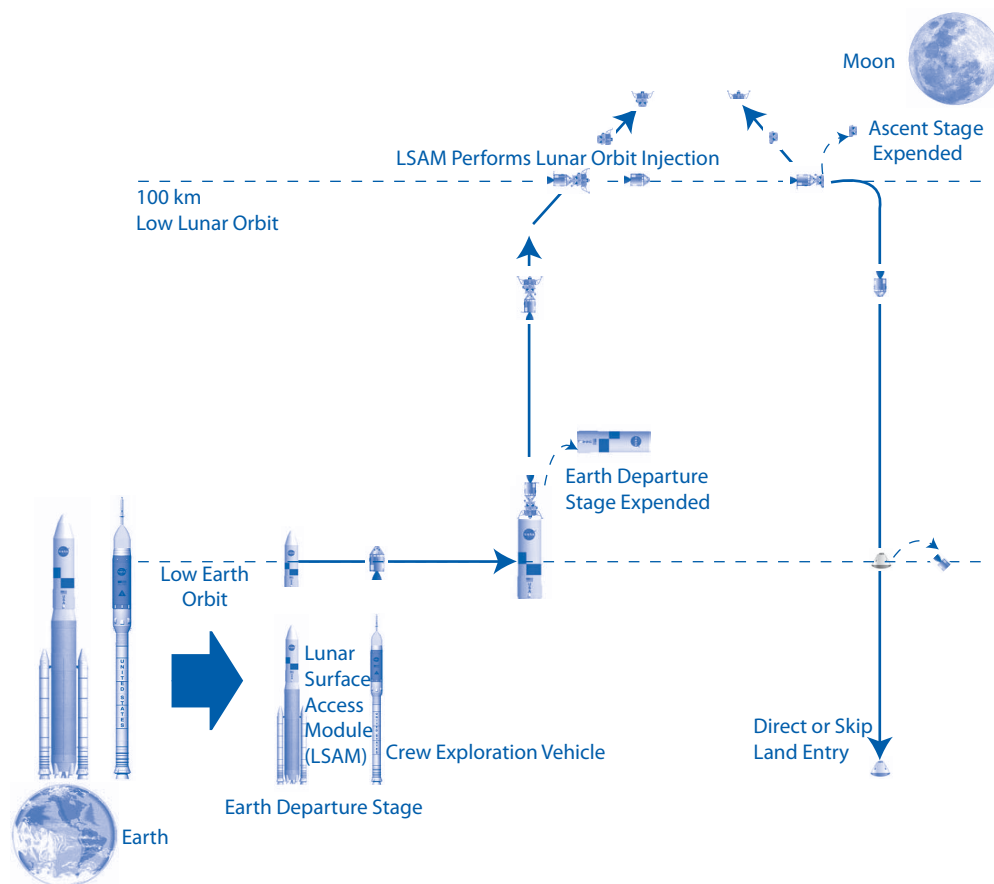


Figure 4.1-5b Example of a lunar sortie DRM early in the life cycle

4.0 System Design

structure for defining the configuration changes and operational activities needed to be carried out to meet the goals of the mission. For each of the operational phases,

facilities, equipment, and critical events should also be included. Table 4.1-1 identifies some common examples of operational phases for a NASA mission.

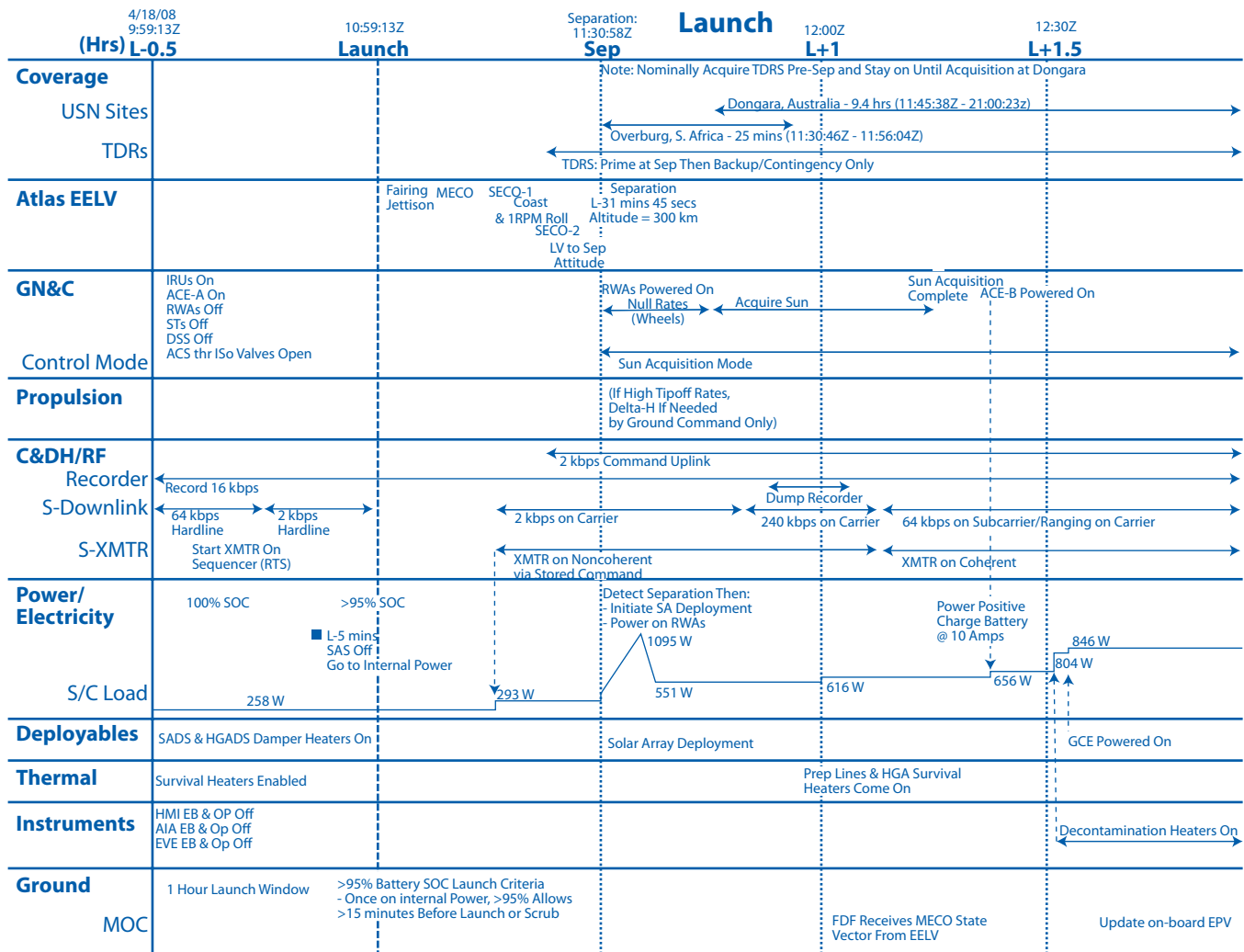


Figure 4.1-6 Example of a more detailed, integrated timeline later in the life cycle for a science mission

Table 4.1-1 Typical Operational Phases for a NASA Mission

Operational Phase	Description
Integration and test operations	Project Integration and Test: During the latter period of project integration and test, the system is tested by performing operational simulations during functional and environmental testing. The simulations typically exercise the end-to-end command and data system to provide a complete verification of system functionality and performance against simulated project operational scenarios.
	Launch Integration: The launch integration phase may repeat integration and test operational and functional verification in the launch-integrated configuration.
Launch operations	Launch: Launch operation occurs during the launch countdown, launch ascent, and orbit injection. Critical event telemetry is an important driver during this phase.
	Deployment: Following orbit injection, spacecraft deployment operations reconfigure the spacecraft to its orbital configuration. Typically, critical events covering solar array, antenna, and other deployments and orbit trim maneuvers occur during this phase.
	In-Orbit Checkout: In-orbit checkout is used to perform a verification that all systems are healthy. This is followed by on-orbit alignment, calibration, and parameterization of the flight systems to prepare for science operations.
Science operations	The majority of the operational lifetime is used to perform science operations.
Safe-hold operations	As a result of on-board fault detection or by ground command, the spacecraft may transition to a safe-hold mode. This mode is designed to maintain the spacecraft in a power positive, thermally stable state until the fault is resolved and science operations can resume.
Anomaly resolution and maintenance operations	Anomaly resolution and maintenance operations occur throughout the mission. They may require resources beyond established operational resources.
Disposal operations	Disposal operations occur at the end of project life. These operations are used to either provide a controlled reentry of the spacecraft or a repositioning of the spacecraft to a disposal orbit. In the latter case, the dissipation of stored fuel and electrical energy is required.

4.2 Technical Requirements Definition

The Technical Requirements Definition Process transforms the stakeholder expectations into a definition of the problem and then into a complete set of validated technical requirements expressed as “shall” statements that can be used for defining a design solution for the Product Breakdown Structure (PBS) model and related enabling products. The process of requirements definition is a recursive and iterative one that develops the stakeholders’ requirements, product requirements, and lower

It is important to note that the team must not rely solely on the requirements received to design and build the system. Communication and iteration with the relevant stakeholders are essential to ensure a mutual understanding of each requirement. Otherwise, the designers run the risk of misunderstanding and implementing an unwanted solution to a different interpretation of the requirements.

level product/component requirements (e.g., PBS model products such as systems or subsystems and related enabling products such as external systems that provide or consume data). The requirements should enable the description of all inputs, outputs, and required relationships between inputs and outputs. The requirements documents organize and communicate requirements to the customer and other stakeholders and the technical community.

Technical requirements definition activities apply to the definition of all technical requirements from the program, project, and system levels down to the lowest level product/component requirements document.

4.2.1 Process Description

Figure 4.2-1 provides a typical flow diagram for the Technical Requirements Definition Process and identifies typical inputs, outputs, and activities to consider in addressing technical requirements definition.

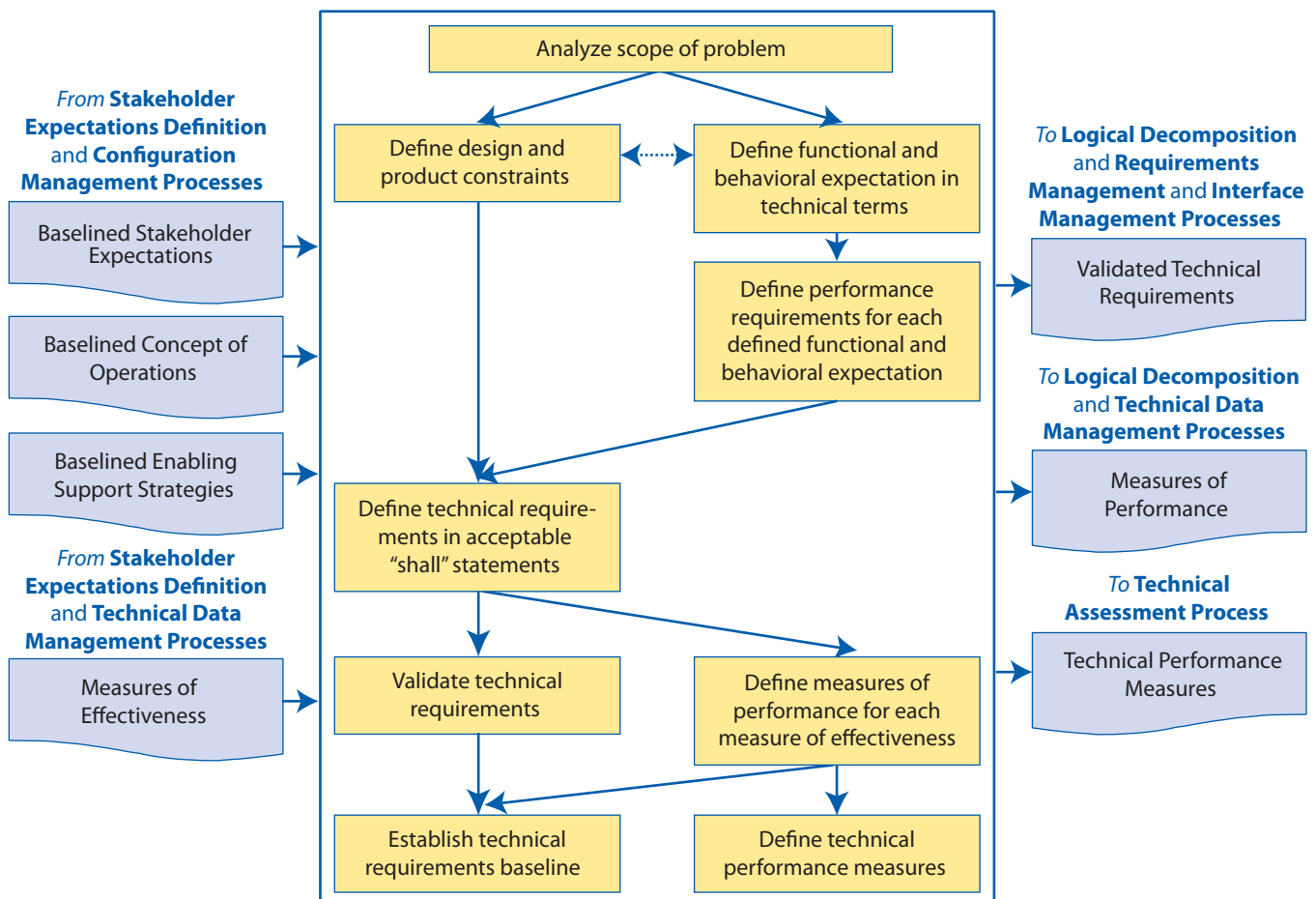


Figure 4.2-1 Technical Requirements Definition Process

4.2.1.1 Inputs

Typical inputs needed for the requirements process would include the following:

- **Top-Level Requirements and Expectations:** These would be the agreed-to top-level requirements and expectations (e.g., needs, wants, desires, capabilities, constraints, external interfaces) for the product(s) to be developed coming from the customer and other stakeholders.
- **Concept of Operations:** This describes how the system will be operated during the life-cycle phases to meet stakeholder expectations. It describes the system characteristics from an operational perspective and helps facilitate an understanding of the system goals. Examples would be a ConOps document or a DRM.

4.2.1.2 Process Activities

The top-level requirements and expectations are initially assessed to understand the technical problem to be solved and establish the design boundary. This boundary is typically established by performing the following activities:

- Defining constraints that the design must adhere to or how the system will be used. The constraints are typically not able to be changed based on tradeoff analyses.
- Identifying those elements that are already under design control and cannot be changed. This helps establish those areas where further trades will be performed to narrow potential design solutions.
- Establishing physical and functional interfaces (e.g., mechanical, electrical, thermal, human, etc.) with which the system must interact.
- Defining functional and behavioral expectations for the range of anticipated uses of the system as identified in the ConOps. The ConOps describes how the system will be operated and the possible use-case scenarios.

With an overall understanding of the constraints, physical/functional interfaces, and functional/behavioral expectations, the requirements can be further defined by establishing performance criteria. The performance is expressed as the quantitative part of the requirement to indicate how well each product function is expected to be accomplished.

Finally, the requirements should be defined in acceptable “shall” statements, which are complete sentences with a single “shall” per statement. See Appendix C for

guidance on how to write good requirements and Appendix E for validating requirements. A well-written requirements document provides several specific benefits to both the stakeholders and the technical team, as shown in Table 4.2-1.

4.2.1.3 Outputs

Typical outputs for the Technical Requirements Definition Process would include the following:

- **Technical Requirements:** This would be the approved set of requirements that represents a complete description of the problem to be solved and requirements that have been validated and approved by the customer and stakeholders. Examples of documentation that capture the requirements are a System Requirements Document (SRD), Project Requirements Document (PRD), Interface Requirements Document (IRD), etc.
- **Technical Measures:** An established set of measures based on the expectations and requirements that will be tracked and assessed to determine overall system or product effectiveness and customer satisfaction. Common terms for these measures are Measures of Effectiveness (MOEs), Measures of Performance (MOPs), and Technical Performance Measures (TPMs). See Section 6.7 for further details.

4.2.2 Technical Requirements Definition Guidance

4.2.2.1 Types of Requirements

A complete set of project requirements includes the functional needs requirements (what functions need to be performed), performance requirements (how well these functions must be performed), and interface requirements (design element interface requirements). For space projects, these requirements are decomposed and allocated down to design elements through the PBS.

Functional, performance, and interface requirements are very important but do not constitute the entire set of requirements necessary for project success. The space segment design elements must also survive and continue to perform in the project environment. These environmental drivers include radiation, thermal, acoustic, mechanical loads, contamination, radio frequency, and others. In addition, reliability requirements drive design choices in design robustness, failure tolerance, and redundancy. Safety requirements drive design choices in providing diverse functional redundancy. Other spe-

Table 4.2-1 Benefits of Well-Written Requirements

Benefit	Rationale
Establish the basis for agreement between the stakeholders and the developers on what the product is to do	The complete description of the functions to be performed by the product specified in the requirements will assist the potential users in determining if the product specified meets their needs or how the product must be modified to meet their needs. During system design, requirements are allocated to subsystems (e.g., hardware, software, and other major components of the system), people, or processes.
Reduce the development effort because less rework is required to address poorly written, missing, and misunderstood requirements	The Technical Requirements Definition Process activities force the relevant stakeholders to consider rigorously all of the requirements before design begins. Careful review of the requirements can reveal omissions, misunderstandings, and inconsistencies early in the development cycle when these problems are easier to correct thereby reducing costly redesign, remanufacture, recoding, and retesting in later life-cycle phases.
Provide a basis for estimating costs and schedules	The description of the product to be developed as given in the requirements is a realistic basis for estimating project costs and can be used to evaluate bids or price estimates.
Provide a baseline for validation and verification	Organizations can develop their validation and verification plans much more productively from a good requirements document. Both system and subsystem test plans and procedures are generated from the requirements. As part of the development, the requirements document provides a baseline against which compliance can be measured. The requirements are also used to provide the stakeholders with a basis for acceptance of the system.
Facilitate transfer	The requirements make it easier to transfer the product to new users or new machines. Stakeholders thus find it easier to transfer the product to other parts of their organization, and developers find it easier to transfer it to new stakeholders or reuse it.
Serve as a basis for enhancement	The requirements serve as a basis for later enhancement or alteration of the finished product.

cialty requirements also may affect design choices. These may include producibility, maintainability, availability, upgradeability, human factors, and others. Unlike functional needs requirements, which are decomposed and allocated to design elements, these requirements are levied across major project elements. Designing to meet these requirements requires careful analysis of design alternatives. Figure 4.2-2 shows the characteristics of functional, operational, reliability, safety, and specialty requirements. Top-level mission requirements are generated from mission objectives, programmatic constraints, and assumptions. These are normally grouped into function and performance requirements and include the categories of requirements in Figure 4.2-2.

Functional Requirements

The functional requirements need to be specified for all intended uses of the product over its entire lifetime. Functional analysis is used to draw out both functional and performance requirements. Requirements are partitioned into groups, based on established criteria (e.g., similar functionality, performance, or coupling, etc.), to facilitate and focus the requirements analysis. Func-

Functional requirements define what functions need to be done to accomplish the objectives.

Performance requirements define how well the system needs to perform the functions.

tional and performance requirements are allocated to functional partitions and subfunctions, objects, people, or processes. Sequencing of time-critical functions is considered. Each function is identified and described in terms of inputs, outputs, and interface requirements from the top down so that the decomposed functions are recognized as part of larger functional groupings. Functions are arranged in a logical sequence so that any specified operational usage of the system can be traced in an end-to-end path to indicate the sequential relationship of all functions that must be accomplished by the system.

It is helpful to walk through the ConOps and scenarios asking the following types of questions: what functions need to be performed, where do they need to be performed, how often, under what operational and environ-



Figure 4.2-2 Characteristics of functional, operational, reliability, safety, and specialty requirements

mental conditions, etc. Thinking through this process often reveals additional functional requirements.

Performance Requirements

Performance requirements quantitatively define how well the system needs to perform the functions. Again, walking through the ConOps and the scenarios often draws out the performance requirements by asking the following types of questions: how often and how well, to what accuracy (e.g., how good does the measurement need to be), what is the quality and quantity of the output, under what stress (maximum simultaneous data

Example of Functional and Performance Requirements

Initial Function Statement

The Thrust Vector Controller (TVC) shall provide vehicle control about the pitch and yaw axes.

This statement describes a high-level function that the TVC must perform. The technical team needs to transform this statement into a set of design-to functional and performance requirements.

Functional Requirements with Associated Performance Requirements

- The TVC shall gimbal the engine a maximum of 9 degrees, ± 0.1 degree.
- The TVC shall gimbal the engine at a maximum rate of 5 degrees/second ± 0.3 degrees/second.
- The TVC shall provide a force of 40,000 pounds, ± 500 pounds.
- The TVC shall have a frequency response of 20 Hz, ± 0.1 Hz.

requests) or environmental conditions, for what duration, at what range of values, at what tolerance, and at what maximum throughput or bandwidth capacity.

Be careful not to make performance requirements too restrictive. For example, for a system that must be able to run on rechargeable batteries, if the performance requirements specify that the time to recharge should be less than 3 hours when a 12-hour recharge time would be sufficient, potential design solutions are eliminated. In the same sense, if the performance requirements specify that a weight must be within ± 0.5 kg, when ± 2.5 kg is sufficient, metrology cost will increase without adding value to the product.

Wherever possible, define the performance requirements in terms of (1) a threshold value (the minimum acceptable value needed for the system to carry out its mission) and (2) the baseline level of performance desired. Specifying performance in terms of thresholds and baseline requirements provides the system designers with trade space in which to investigate alternative designs.

All qualitative performance expectations must be analyzed and translated into quantified performance requirements. Trade studies often help quantify performance requirements. For example, tradeoffs can show whether

4.0 System Design

a slight relaxation of the performance requirement could produce a significantly cheaper system or whether a few more resources could produce a significantly more effective system. The rationale for thresholds and goals should be documented with the requirements to understand the reason and origin for the performance requirement in case it must be changed. The performance requirements that can be quantified by or changed by tradeoff analysis should be identified. See Section 6.8, Decision Analysis, for more information on tradeoff analysis.

Interface Requirements

It is important to define all interface requirements for the system, including those to enabling systems. The external interfaces form the boundaries between the product and the rest of the world. Types of interfaces include: operational command and control, computer to computer, mechanical, electrical, thermal, and data. One useful tool in defining interfaces is the context diagram (see Appendix F), which depicts the product and all of its external interfaces. Once the product components are defined, a block diagram showing the major components, interconnections, and external interfaces of the system should be developed to define both the components and their interactions.

Interfaces associated with all product life-cycle phases should also be considered. Examples include interfaces with test equipment; transportation systems; Integrated Logistics Support (ILS) systems; and manufacturing facilities, operators, users, and maintainers.

As the technical requirements are defined, the interface diagram should be revisited and the documented interface requirements refined to include newly identified interfaces information for requirements both internal and external. More information regarding interfaces can be found in Section 6.3.

Environmental Requirements

Each space mission has a unique set of environmental requirements that apply to the flight segment elements. It is a critical function of systems engineering to identify the external and internal environments for the particular mission, analyze and quantify the expected environments, develop design guidance, and establish a margin philosophy against the expected environments.

The environments envelope should consider what can be encountered during ground test, storage, transportation,

launch, deployment, and normal operations from beginning of life to end of life. Requirements derived from the mission environments should be included in the system requirements.

External and internal environment concerns that must be addressed include acceleration, vibration, shock, static loads, acoustic, thermal, contamination, crew-induced loads, total dose radiation/radiation effects, Single-Event Effects (SEEs), surface and internal charging, orbital debris, atmospheric (atomic oxygen) control and quality, attitude control system disturbance (atmospheric drag, gravity gradient, and solar pressure), magnetic, pressure gradient during launch, microbial growth, and radio frequency exposure on the ground and on orbit.

The requirements structure must address the specialty engineering disciplines that apply to the mission environments across project elements. These discipline areas levy requirements on system elements regarding Electromagnetic Interference, Electromagnetic Compatibility (EMI/EMC), grounding, radiation and other shielding, contamination protection, and reliability.

Reliability Requirements

Reliability can be defined as the probability that a device, product, or system will not fail for a given period of time under specified operating conditions. Reliability is an inherent system design characteristic. As a principal contributing factor in operations and support costs and in system effectiveness, reliability plays a key role in determining the system's cost-effectiveness.

Reliability engineering is a major specialty discipline that contributes to the goal of a cost-effective system. This is primarily accomplished in the systems engineering process through an active role in implementing specific design features to ensure that the system can perform in the predicted physical environments throughout the mission, and by making independent predictions of system reliability for design trades and for test program, operations, and integrated logistics support planning.

Reliability requirements ensure that the system (and subsystems, e.g., software and hardware) can perform in the predicted environments and conditions as expected throughout the mission and that the system has the ability to withstand certain numbers and types of faults, errors, or failures (e.g., withstand vibration, predicted data rates, command and/or data errors, single-event

upsets, and temperature variances to specified limits). Environments can include ground (transportation and handling), launch, on-orbit (Earth or other), planetary, reentry, and landing, or they might be for software within certain modes or states of operation. Reliability addresses design and verification requirements to meet the requested level of operation as well as fault and/or failure tolerance for all expected environments and conditions. Reliability requirements cover fault/failure prevention, detection, isolation, and recovery.

Safety Requirements

NASA uses the term “safety” broadly to include human (public and workforce), environmental, and asset safety. There are two types of safety requirements—deterministic and risk-informed. A deterministic safety requirement is the qualitative or quantitative definition of a threshold of action or performance that must be met by a mission-related design item, system, or activity for that item, system, or activity to be acceptably safe. Examples of deterministic safety requirements are incorporation of safety devices (e.g., build physical hardware stops into the system to prevent the hydraulic lift/arm from extending past allowed safety height and length limits); limits on the range of values a system input variable is allowed to take on; and limit checks on input commands to ensure they are within specified safety limits or constraints for that mode or state of the system (e.g., the command to retract the landing gear is only allowed if the airplane is in the airborne state). For those components identified as “safety critical,” requirements include functional redundancy or failure tolerance to allow the system to meet its requirements in the presence of one or more failures or to take the system to a safe state with reduced functionality (e.g., dual redundant computer processors, safe-state backup processor); detection and automatic system shutdown if specified values (e.g., temperature) exceed prescribed safety limits; use of only a subset that is approved for safety-critical software of a particular computer language; caution or warning devices; and safety procedures. A risk-informed safety requirement is a requirement that has been established, at least in part, on the basis of the consideration of safety-related TPMs and their associated uncertainty. An example of a risk-informed safety requirement is the Probability of Loss of Crew (P(LOC)) not exceeding a certain value “p” with a certain confidence level. Meeting safety requirements involves identification and elimination of hazards, reducing the likelihood of the accidents associated with hazards, or reducing

the impact from the hazard associated with these accidents to within acceptable levels. (For additional information concerning safety, see, for example, *NPR 8705.2, Human-Rating Requirements for Space Systems*, *NPR 8715.3, NASA General Safety Program Requirements*, and *NASA-STD-8719.13, Software Safety Standard*.)

4.2.2.2 Human Factors Engineering Requirements

In human spaceflight, the human—as operator and as maintainer—is a critical component of the mission and system design. Human capabilities and limitations must enter into designs in the same way that the properties of materials and characteristics of electronic components do. Human factors engineering is the discipline that studies human-system interfaces and interactions and provides requirements, standards, and guidelines to ensure the entire system can function as designed with effective accommodation of the human component.

Humans are initially integrated into systems through analysis of the overall mission. Mission functions are allocated to humans as appropriate to the system architecture, technical capabilities, cost factors, and crew capabilities. Once functions are allocated, human factors analysts work with system designers to ensure that human operators and maintainers are provided the equipment, tools, and interfaces to perform their assigned tasks safely and effectively.

NASA-STD-3001, NASA Space Flight Human System Standards Volume 1: Crew Health ensures that systems are safe and effective for humans. The standards focus on the human integrated with the system, the measures needed (rest, nutrition, medical care, exercise, etc.) to ensure that the human stays healthy and effective, the workplace environment, and crew-system physical and cognitive interfaces.

4.2.2.3 Requirements Decomposition, Allocation, and Validation

Requirements are decomposed in a hierarchical structure starting with the highest level requirements imposed by Presidential directives, mission directorates, program, Agency, and customer and other stakeholders. These high-level requirements are decomposed into functional and performance requirements and allocated across the system. These are then further decomposed and allocated among the elements and subsystems. This

4.0 System Design

decomposition and allocation process continues until a complete set of design-to requirements is achieved. At each level of decomposition (system, subsystem, component, etc.), the total set of derived requirements must be validated against the stakeholder expectations or higher level parent requirements before proceeding to the next level of decomposition.

The traceability of requirements to the lowest level ensures that each requirement is necessary to meet the stakeholder expectations. Requirements that are not allocated to lower levels or are not implemented at a lower level result in a design that does not meet objectives and is, therefore, not valid. Conversely, lower level requirements that are not traceable to higher level requirements

result in an overdesign that is not justified. This hierarchical flowdown is illustrated in Figure 4.2-3.

Figure 4.2-4 is an example of how science pointing requirements are successively decomposed and allocated from the top down for a typical science mission. It is important to understand and document the relationship between requirements. This will reduce the possibility of misinterpretation and the possibility of an unsatisfactory design and associated cost increases.

Throughout Phases A and B, changes in requirements and constraints will occur. It is imperative that all changes be thoroughly evaluated to determine the impacts on both higher and lower hierarchical levels. All changes must be

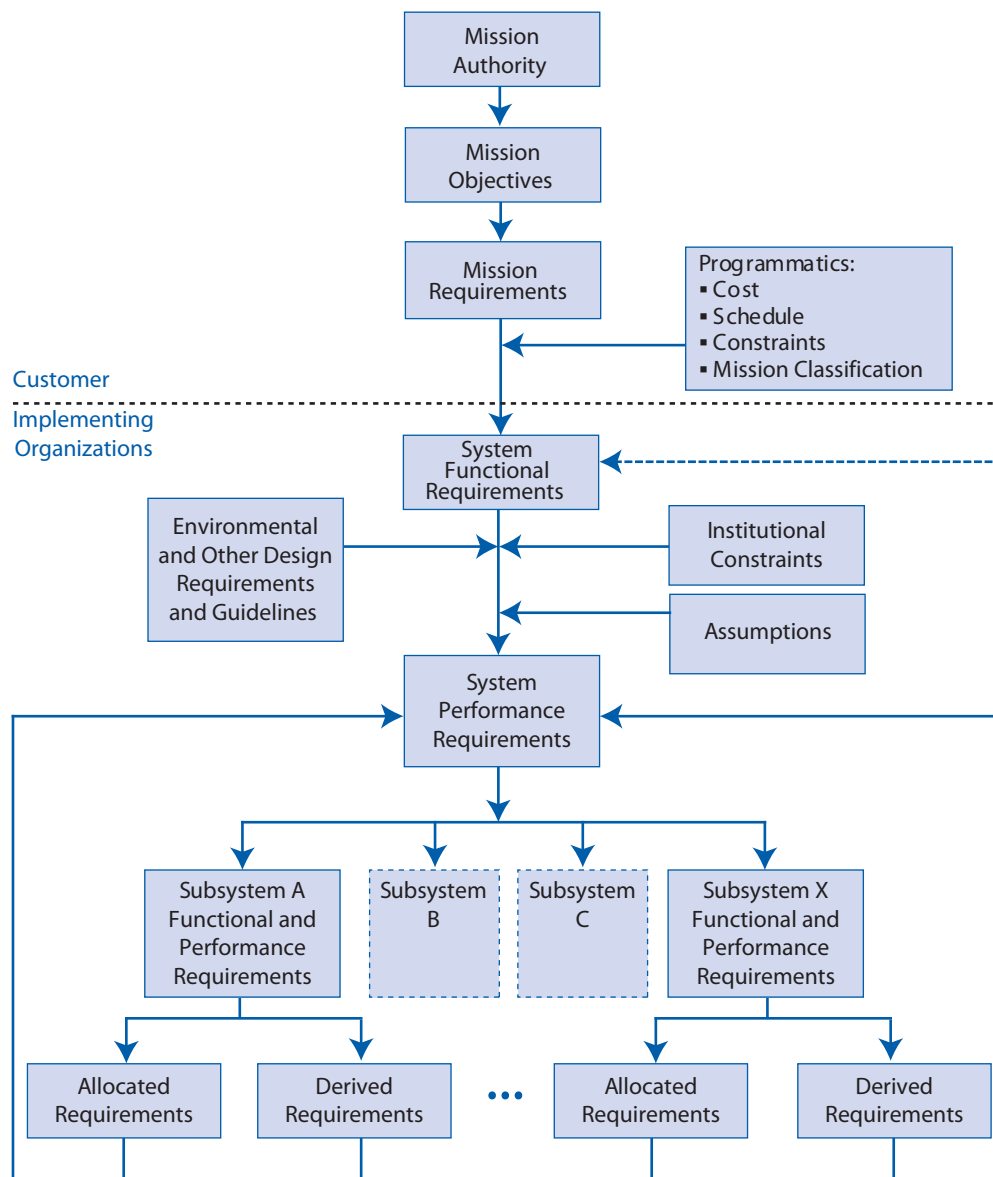


Figure 4.2-3 The flowdown of requirements

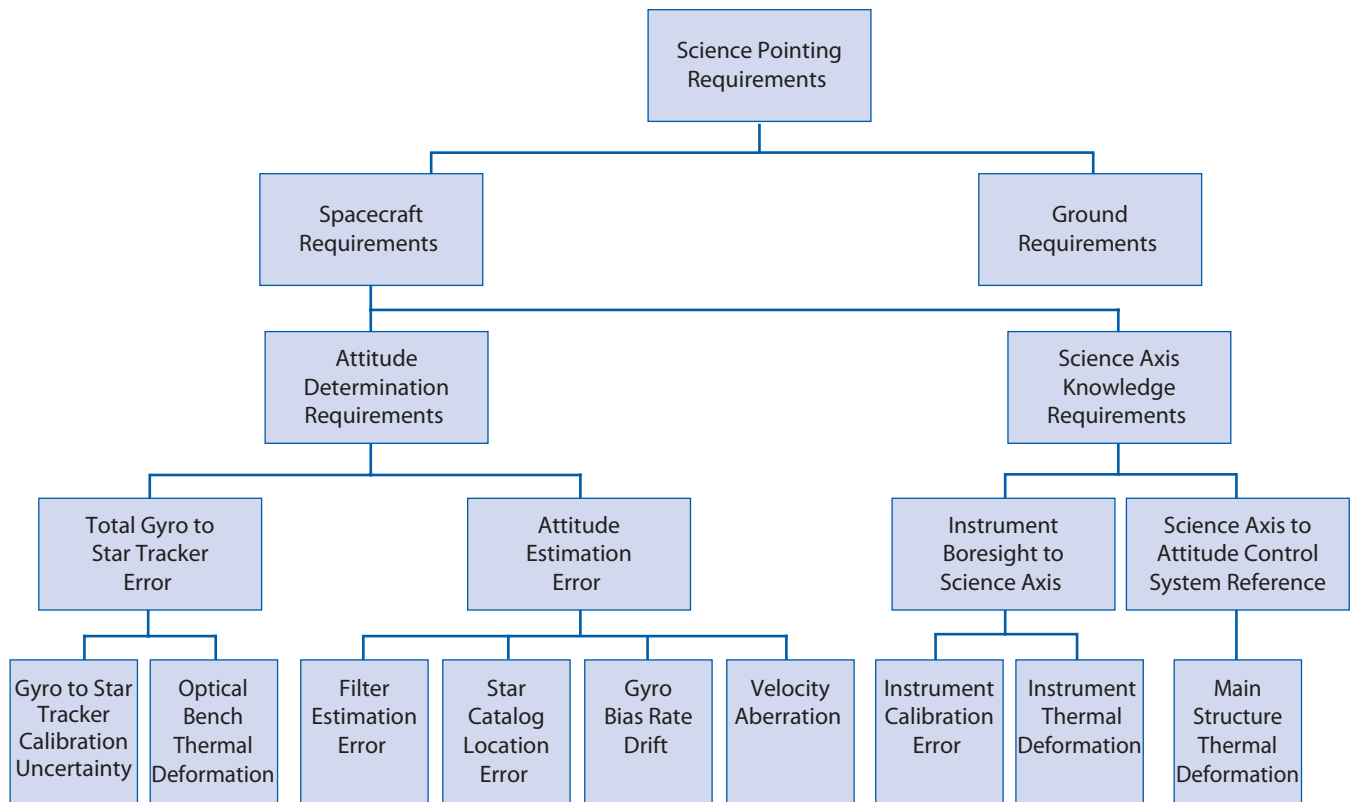


Figure 4.2-4 Allocation and flowdown of science pointing requirements

subjected to a review and approval cycle as part of a formal change control process to maintain traceability and to ensure the impacts of any changes are fully assessed for all parts of the system. A more formal change control process is required if the mission is very large and involves more than one Center or crosses other jurisdictional or organizational boundaries.

4.2.2.4 Capturing Requirements and the Requirements Database

At the time the requirements are written, it is important to capture the requirements statements along with the metadata associated with each requirement. The metadata is the supporting information necessary to help clarify and link the requirements.

The method of verification must also be thought through and captured for each requirement at the time it is developed. The verification method includes test, inspection, analysis, and demonstration. Be sure to document any new or derived requirements that are uncovered during determination of the verification method. An example is requiring an additional test port to give visibility to an internal signal during integration and test. If a requirement cannot be verified, then either it

should not be a requirement or the requirement statement needs to be rewritten. For example, the requirement to “minimize noise” is vague and cannot be verified. If the requirement is restated as “the noise level of the component X shall remain under Y decibels” then it is clearly verifiable. Examples of the types of metadata are provided in Table 4.2-2.

The requirements database is an extremely useful tool for capturing the requirements and the associated metadata and for showing the bidirectional traceability between requirements. The database evolves over time and could be used for tracking status information related to requirements such as To Be Determined (TBD)/To Be Resolved (TBR) status, resolution date, and verification status. Each project should decide what metadata will be captured. The database is usually in a central location that is made available to the entire project team. (See Appendix D for a sample requirements verification matrix.)

4.2.2.5 Technical Standards

Importance of Standards Application

Standards provide a proven basis for establishing common technical requirements across a program or

Table 4.2-2 Requirements Metadata

Item	Function
Requirement ID	Provides a unique numbering system for sorting and tracking.
Rationale	Provides additional information to help clarify the intent of the requirements at the time they were written. (See “Rationale” box below on what should be captured.)
Traced from	Captures the bidirectional traceability between parent requirements and lower level (derived) requirements and the relationships between requirements.
Owner	Person or group responsible for writing, managing, and/or approving changes to this requirement.
Verification method	Captures the method of verification (test, inspection, analysis, demonstration) and should be determined as the requirements are developed.
Verification lead	Person or group assigned responsibility for verifying the requirement.
Verification level	Specifies the level in the hierarchy at which the requirements will be verified (e.g., system, subsystem, element).

Rationale

The rationale should be kept up to date and include the following information:

- **Reason for the Requirement:** Often the reason for the requirement is not obvious, and it may be lost if not recorded as the requirement is being documented. The reason may point to a constraint or concept of operations. If there is a clear parent requirement or trade study that explains the reason, then reference it.
- **Document Assumptions:** If a requirement was written assuming the completion of a technology development program or a successful technology mission, document the assumption.
- **Document Relationships:** The relationships with the product’s expected operations (e.g., expectations about how stakeholders will use a product). This may be done with a link to the ConOps.
- **Document Design Constraints:** Imposed by the results from decisions made as the design evolves. If the requirement states a method of implementation, the rationale should state why the decision was made to limit the solution to this one method of implementation.

project to avoid incompatibilities and ensure that at least minimum requirements are met. Common standards can also lower implementation cost as well as costs for inspection, common supplies, etc. Typically, standards (and specifications) are used throughout the product life cycle to establish design requirements and margins, materials and process specifications, test methods, and interface specifications. Standards are used as requirements (and guidelines) for design, fabrication, verification, validation, acceptance, operations, and maintenance.

Selection of Standards

NASA policy for technical standards is provided in *NPD 8070.6, Technical Standards*, which addresses selection, tailoring, application, and control of standards. In gen-

eral, the order of authority among standards for NASA programs and projects is as follows:

- Standards mandated by law (e.g., environmental standards),
- National or international voluntary consensus standards recognized by industry,
- Other Government standards,
- NASA policy directives, and
- NASA technical standards.

NASA may also designate mandatory or “core” standards that must be applied to all programs where technically applicable. Waivers to designated core standards must be justified and approved at the Agency level unless otherwise delegated.

4.3 Logical Decomposition

Logical Decomposition is the process for creating the detailed functional requirements that enable NASA programs and projects to meet the stakeholder expectations. This process identifies the “what” that must be achieved by the system at each level to enable a successful project. Logical decomposition utilizes functional analysis to create a system architecture and to decompose top-level (or parent) requirements and allocate them down to the lowest desired levels of the project.

The Logical Decomposition Process is used to:

- Improve understanding of the defined technical requirements and the relationships among the requirements (e.g., functional, behavioral, and temporal), and
- Decompose the parent requirements into a set of logical decomposition models and their associated sets of derived technical requirements for input to the Design Solution Definition Process.

4.3.1 Process Description

Figure 4.3-1 provides a typical flow diagram for the Logical Decomposition Process and identifies typical inputs, outputs, and activities to consider in addressing logical decomposition.

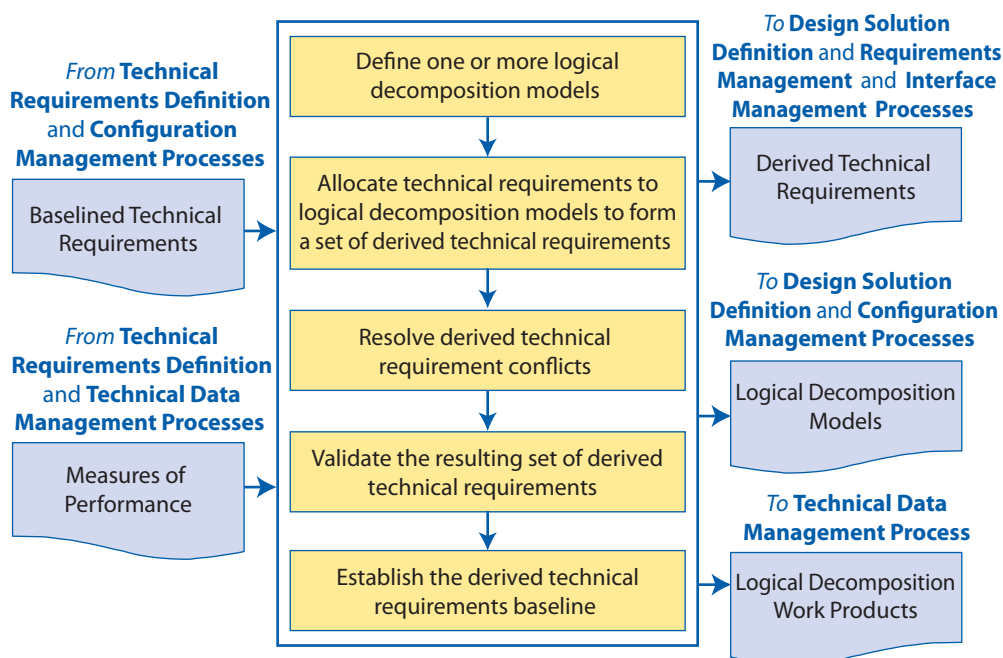


Figure 4.3-1 Logical Decomposition Process

4.3.1.1 Inputs

Typical inputs needed for the Logical Decomposition Process would include the following:

- **Technical Requirements:** A validated set of requirements that represent a description of the problem to be solved, have been established by functional and performance analysis, and have been approved by the customer and other stakeholders. Examples of documentation that capture the requirements are an SRD, PRD, and IRD.
- **Technical Measures:** An established set of measures based on the expectations and requirements that will be tracked and assessed to determine overall system or product effectiveness and customer satisfaction. These measures are MOEs, MOPs, and a special subset of these called TPMs. See Subsection 6.7.2.2 for further details.

4.3.1.2 Process Activities

The key first step in the Logical Decomposition Process is establishing the system architecture model. The system architecture activity defines the underlying structure and relationships of hardware, software, communications, operations, etc., that provide for the implementation of Agency, mission directorate, program, project,

and subsequent levels of the requirements. System architecture activities drive the partitioning of system elements and requirements to lower level functions and requirements to the point that design work can be accomplished. Interfaces and relationships between partitioned subsystems and elements are defined as well.

Once the top-level (or parent) functional requirements and constraints have been established, the system designer uses functional analysis to begin to formulate a conceptual system architecture. The system ar-

4.0 System Design

Architecture can be seen as the strategic organization of the functional elements of the system, laid out to enable the roles, relationships, dependencies, and interfaces between elements to be clearly defined and understood. It is strategic in its focus on the overarching structure of the system and how its elements fit together to contribute to the whole, instead of on the particular workings of the elements themselves. It enables the elements to be developed separately from each other while ensuring that they work together effectively to achieve the top-level (or parent) requirements.

Much like the other elements of functional decomposition, the development of a good system-level architecture is a creative, recursive, and iterative process that combines an excellent understanding of the project's end objectives and constraints with an equally good knowledge of various potential technical means of delivering the end products.

Focusing on the project's ends, top-level (or parent) requirements, and constraints, the system architect must develop at least one, but preferably multiple, concept architectures capable of achieving program objectives. Each architecture concept involves specification of the functional elements (what the pieces do), their relationships to each other (interface definition), and the ConOps, i.e., how the various segments, subsystems, elements, units, etc., will operate as a system when distributed by location and environment from the start of operations to the end of the mission.

The development process for the architectural concepts must be recursive and iterative, with feedback from stakeholders and external reviewers, as well as from subsystem designers and operators, provided as often as possible to increase the likelihood of achieving the program's ends, while reducing the likelihood of cost and schedule overruns.

In the early stages of the mission, multiple concepts are developed. Cost and schedule constraints will ultimately limit how long a program or project can maintain multiple architectural concepts. For all NASA programs, architecture design is completed during the Formulation phase. For most NASA projects (and tightly coupled programs), the selection of a single architecture will happen during Phase A, and the architecture and ConOps will be baselined during Phase B. Architectural changes at higher levels occasionally occur as decomposition to

lower levels produces complications in design, cost, or schedule that necessitate such changes.

Aside from the creative minds of the architects, there are multiple tools that can be utilized to develop a system's architecture. These are primarily modeling and simulation tools, functional analysis tools, architecture frameworks, and trade studies. (For example, one way of doing architecture is the Department of Defense (DOD) Architecture Framework (DODAF). See box.) As each concept is developed, analytical models of the architecture, its elements, and their operations will be developed with increased fidelity as the project evolves. Functional decomposition, requirements development, and trade studies are subsequently undertaken. Multiple iterations of these activities feed back to the evolving architectural concept as the requirements flow down and the design matures.

Functional analysis is the primary method used in system architecture development and functional requirement decomposition. It is the systematic process of identifying, describing, and relating the functions a system must perform to fulfill its goals and objectives. Functional analysis identifies and links system functions, trade studies, interface characteristics, and rationales to requirements. It is usually based on the ConOps for the system of interest.

Three key steps in performing functional analysis are:

- Translate top-level requirements into functions that must be performed to accomplish the requirements.
- Decompose and allocate the functions to lower levels of the product breakdown structure.
- Identify and describe functional and subsystem interfaces.

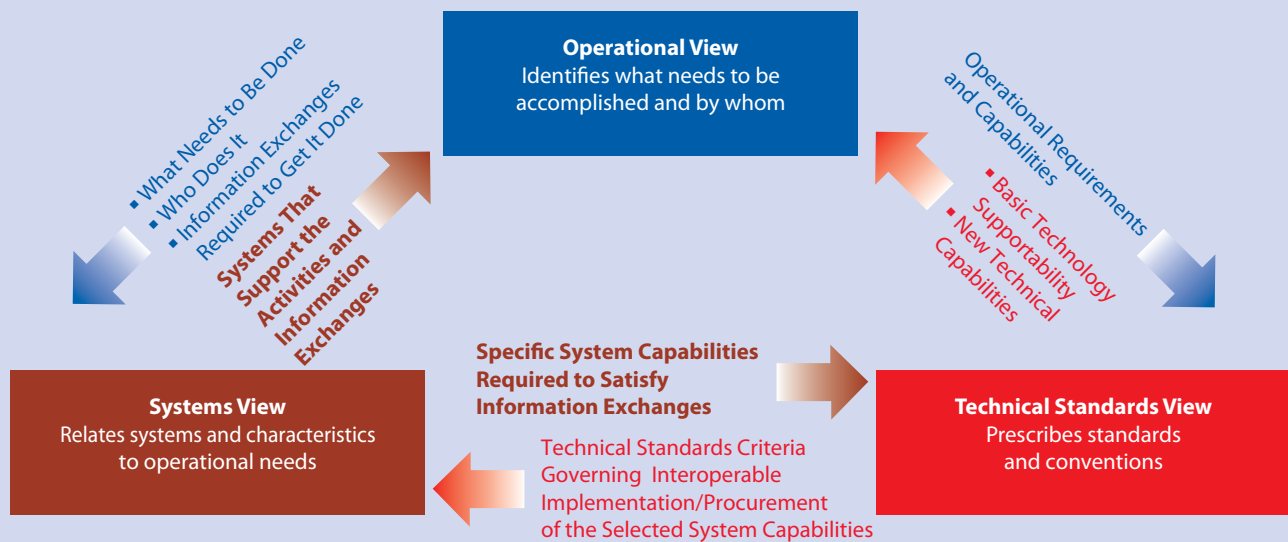
The process involves analyzing each system requirement to identify all of the functions that must be performed to meet the requirement. Each function identified is described in terms of inputs, outputs, and interface requirements. The process is repeated from the top down so that subfunctions are recognized as part of larger functional areas. Functions are arranged in a logical sequence so that any specified operational usage of the system can be traced in an end-to-end path.

The process is recursive and iterative and continues until all desired levels of the architecture/system have been analyzed, defined, and baselined. There will almost cer-

DOD Architecture Framework

New ways, called architecture frameworks, have been developed in the last decade to describe and characterize evolving, complex system-of-systems. In such circumstances, architecture descriptions are very useful in ensuring that stakeholder needs are clearly understood and prioritized, that critical details such as interoperability are addressed upfront, and that major investment decisions are made strategically. In recognition of this, the U.S. Department of Defense has established policies that mandate the use of the DODAF in capital planning, acquisition, and joint capabilities integration.

An architecture can be understood as “the structure of components, their relationships, and the principles and guidelines governing their design and evolution over time.”* To describe an architecture, the DODAF defines several views: operational, systems, and technical standards. In addition, a dictionary and summary information are also required. (See figure below.)



Within each of these views, DODAF contains specific *products*. For example, within the Operational View is a description of the operational nodes, their connectivity, and information exchange requirements. Within the Systems View is a description of all the systems contained in the operational nodes and their interconnectivity. Not all DODAF products are relevant to NASA systems engineering, but its underlying concepts and formalisms may be useful in structuring complex problems for the Technical Requirements Definition and Decision Analysis Processes.

*Definition based on Institute of Electrical and Electronics Engineers (IEEE) STD 610.12.

Source: DOD, *DOD Architecture Framework*.

tainly be alternative ways to decompose functions; therefore, the outcome is highly dependent on the creativity, skills, and experience of the engineers doing the analysis. As the analysis proceeds to lower levels of the architecture and system and the system is better understood, the systems engineer must keep an open mind and a willingness to go back and change previously established architecture and system requirements. These changes will then have to be decomposed down through the architecture and systems again, with the recursive process con-

tinuing until the system is fully defined, with all of the requirements understood and known to be viable, verifiable, and internally consistent. Only at that point should the system architecture and requirements be baselined.

4.3.1.3 Outputs

Typical outputs of the Logical Decomposition Process would include the following:

- **System Architecture Model:** Defines the underlying structure and relationship of the elements of the

4.0 System Design

system (e.g., hardware, software, communications, operations, etc.) and the basis for the partitioning of requirements into lower levels to the point that design work can be accomplished.

- **End Product Requirements:** A defined set of make-to, buy-to, code-to, and other requirements from which design solutions can be accomplished.

4.3.2 Logical Decomposition Guidance

4.3.2.1 Product Breakdown Structure

The decompositions represented by the PBS and the Work Breakdown Structure (WBS) form important perspectives on the desired product system. The WBS is a hierarchical breakdown of the work necessary to complete the project. See Subsection 6.1.2.1 for further information on WBS development. The WBS contains the PBS, which is the hierarchical breakdown of the products such as hardware items, software items, and information items (documents, databases, etc.). The PBS is used during the Logical Decomposition and functional analysis processes. The PBS should be carried down to the lowest level for which there is a cognizant engineer or manager. Figure 4.3-2 is an example of a PBS.

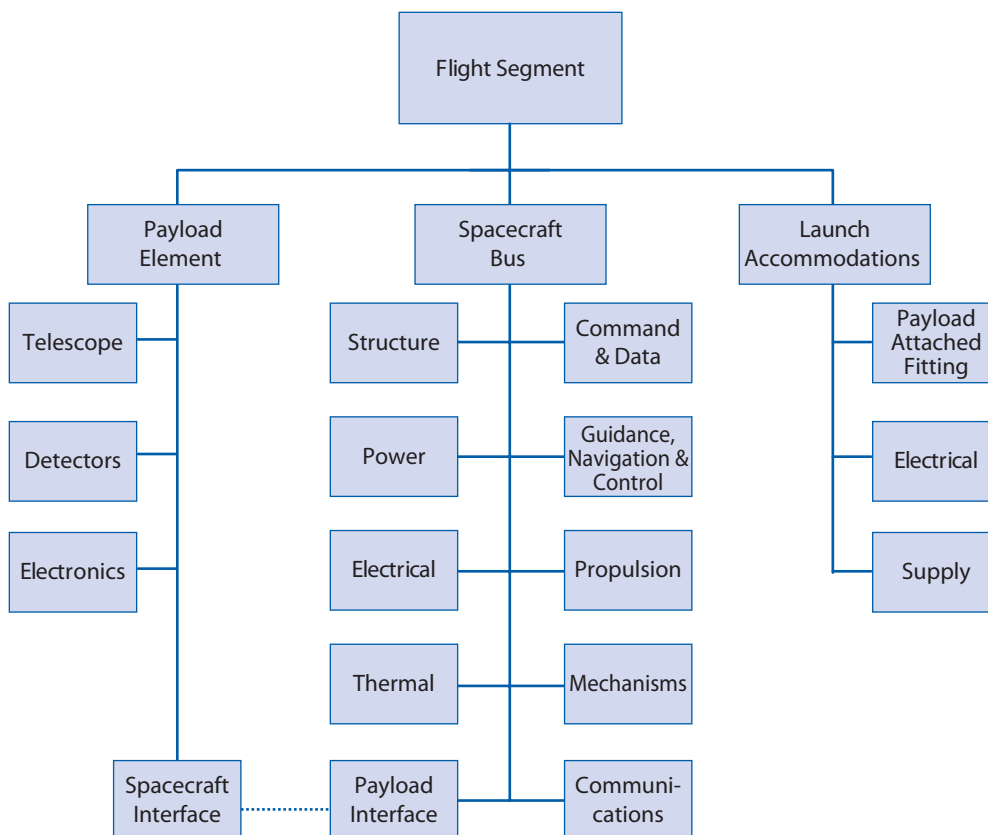


Figure 4.3-2 Example of a PBS

4.3.2.2 Functional Analysis Techniques

Although there are many techniques available to perform functional analysis, some of the more popular are (1) Functional Flow Block Diagrams (FFBDs) to depict task sequences and relationships, (2) N2 diagrams (or $N \times N$ interaction matrix) to identify interactions or interfaces between major factors from a systems perspective, and (3) Timeline Analyses (TLAs) to depict the time sequence of time-critical functions.

Functional Flow Block Diagrams

The primary functional analysis technique is the functional flow block diagram. The purpose of the FFBD is to indicate the sequential relationship of all functions that must be accomplished by a system. When completed, these diagrams show the entire network of actions that lead to the fulfillment of a function.

FFBDs specifically depict each functional event (represented by a block) occurring following the preceding function. Some functions may be performed in parallel, or alternative paths may be taken. The FFBD network shows the logical sequence of “what” must happen; it does not ascribe a time duration to functions or between

functions. The duration of the function and the time between functions may vary from a fraction of a second to many weeks. To understand *time-critical* requirements, a TLA is used. (See the TLA discussion later in this subsection.)

The FFBDs are function oriented, not equipment oriented. In other words, they identify “what” must happen and must not assume a particular answer to “how” a function will be performed. The “how” is then defined for each block at a given level by defining the “what” functions at the next lower level necessary to accomplish that block. In this way, FFBDs are developed from the top down,

in a series of levels, with tasks at each level identified through functional decomposition of a single task at a higher level. The FFBD displays all of the tasks at each level in their logical, sequential relationship, with their required inputs and anticipated outputs (including metrics, if applicable), plus a clear link back to the single, higher level task.

An example of an FFBD is shown in Figure 4.3-3. The FFBD depicts the entire flight mission of a spacecraft.

Each block in the first level of the diagram is expanded to a series of functions, as shown in the second-level diagram for “Perform Mission Operations.” Note that the diagram shows both input (“Transfer to OPS Orbit”) and output (“Transfer to STS Orbit”), thus initiating the interface identification and control process. Each block in the second-level diagram can be progressively developed into a series of functions, as shown in the third-level diagram.

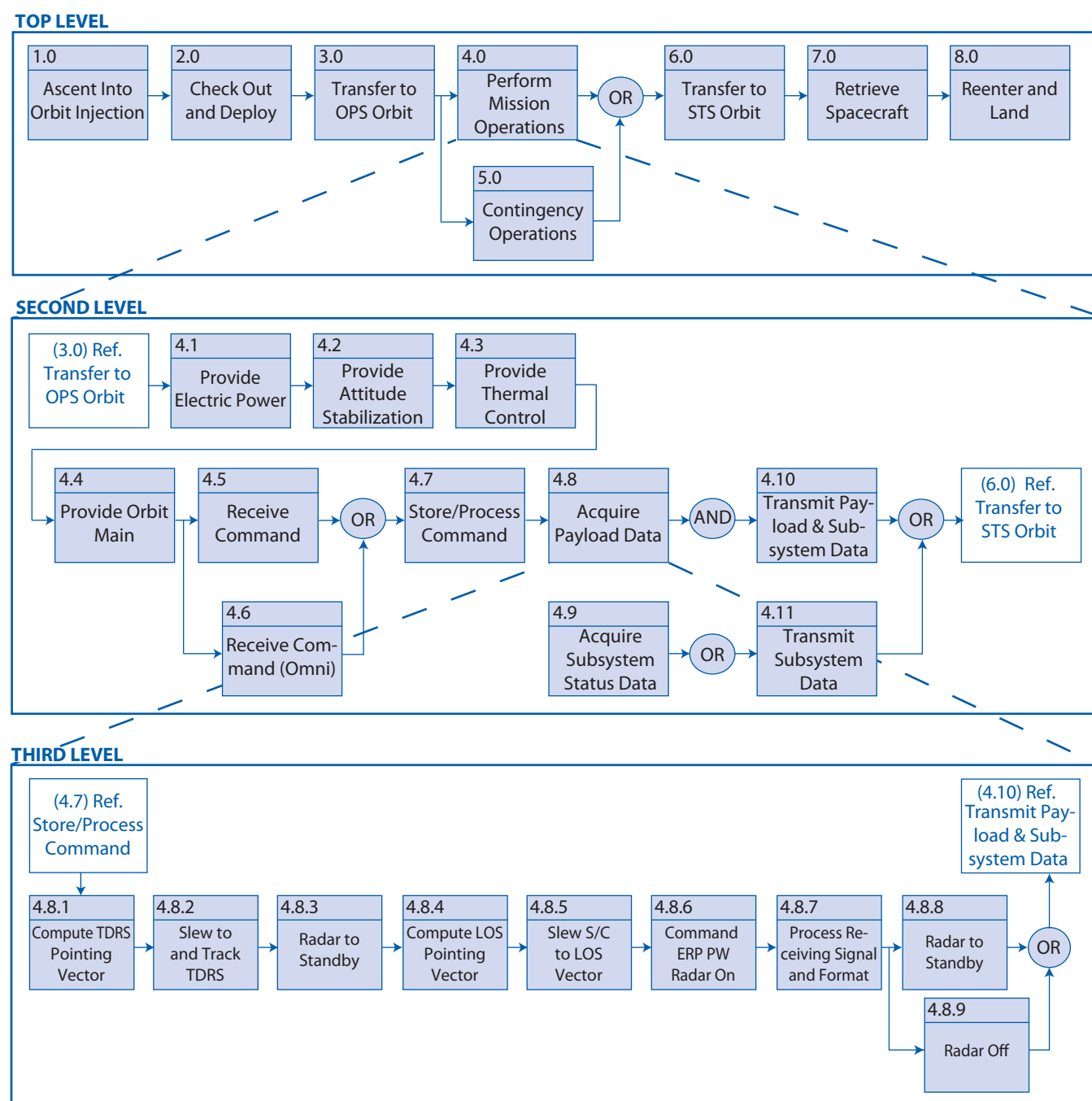


Figure 4.3-3 Example of a functional flow block diagram

4.0 System Design

FFBDs are used to develop, analyze, and flow down requirements, as well as to identify profitable trade studies, by identifying alternative approaches to performing each function. In certain cases, alternative FFBDs may be used to represent various means of satisfying a particular function until trade study data are acquired to permit selection among the alternatives.

The flow diagram also provides an understanding of the total operation of the system, serves as a basis for development of operational and contingency procedures, and pinpoints areas where changes in operational procedures could simplify the overall system operation.

N2 Diagrams

The N-squared (N^2 or N^2) diagram is used to develop system interfaces. An example of an N^2 diagram is shown in Figure 4.3-4. The system components or functions are placed on the diagonal; the remainder of

the squares in the $N \times N$ matrix represent the interface inputs and outputs. Where a blank appears, there is no interface between the respective components or functions. The N^2 diagram can be taken down into successively lower levels to the component functional levels. In addition to defining the interfaces, the N^2 diagram also pinpoints areas where conflicts could arise in interfaces, and highlights input and output dependency assumptions and requirements.

Timeline Analysis

TLA adds consideration of functional durations and is performed on those areas where time is critical to mission success, safety, utilization of resources, minimization of downtime, and/or increasing availability. TLA can be applied to such diverse operational functions as spacecraft command sequencing and launch; but for those functional sequences where time is not a critical factor, FFBDs or N^2 diagrams are sufficient. The following areas are often categorized as time-critical: (1) functions affecting system

reaction time, (2) mission turnaround time, (3) time countdown activities, and (4) functions for which optimum equipment and/or personnel utilization are dependent on the timing of particular activities.

Timeline Sheets (TLs) are used to perform and record the analysis of time-critical functions and functional sequences. For time-critical functional sequences, the time requirements are specified with associated tolerances. Additional tools such as mathematical models and computer simulations may be necessary to establish the duration of each timeline.

For additional information on FFBD, N^2 diagrams, timeline analysis, and other functional analysis methods, see Appendix F.

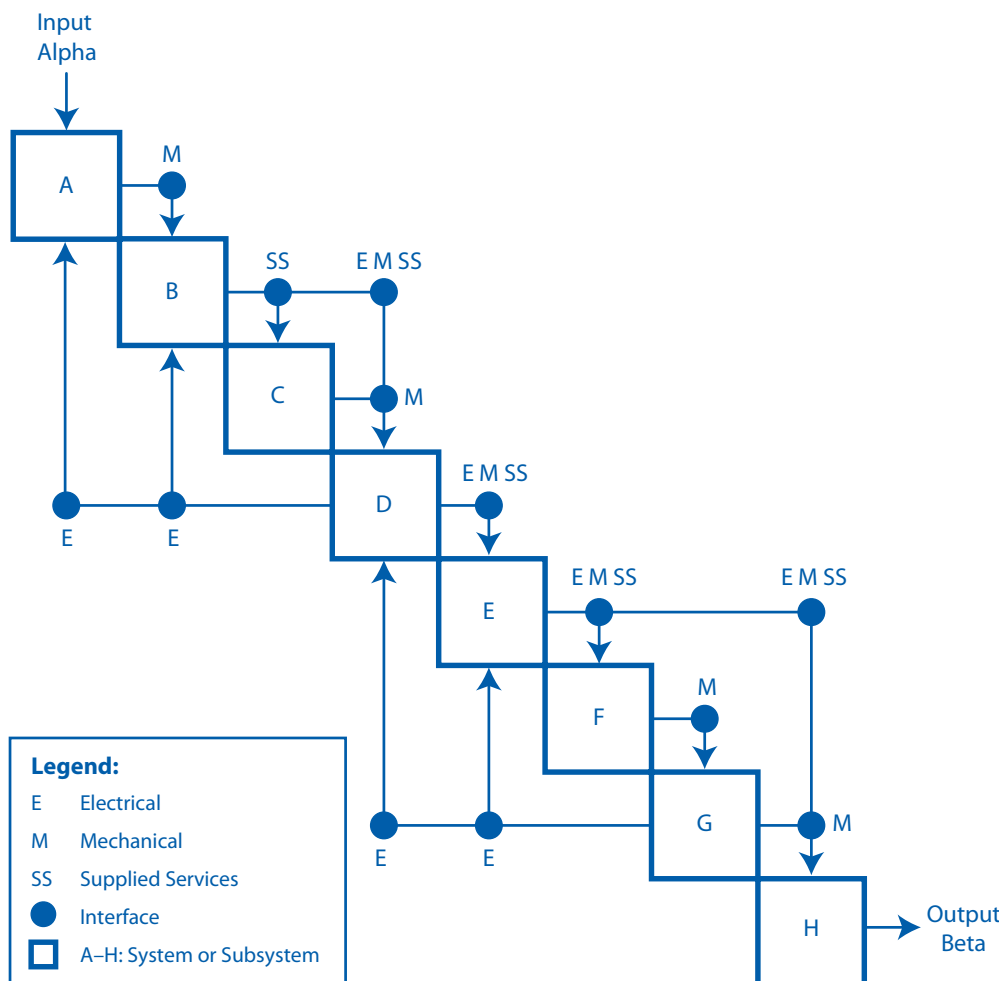


Figure 4.3-4 Example of an N^2 diagram

4.4 Design Solution Definition

The Design Solution Definition Process is used to translate the high-level requirements derived from the stakeholder expectations and the outputs of the Logical Decomposition Process into a design solution. This involves transforming the defined logical decomposition models and their associated sets of derived technical requirements into alternative solutions. These alternative solutions are then analyzed through detailed trade studies that result in the selection of a preferred alternative. This preferred alternative is then fully defined into a final design solution that will satisfy the technical requirements. This design solution definition will be used to generate the end product specifications that will be used to produce the product and to conduct product verification. This process may be further refined depending on whether

there are additional subsystems of the end product that need to be defined.

4.4.1 Process Description

Figure 4.4-1 provides a typical flow diagram for the Design Solution Definition Process and identifies typical inputs, outputs, and activities to consider in addressing design solution definition.

4.4.1.1 Inputs

There are several fundamental inputs needed to initiate the Design Solution Definition Process:

- **Technical Requirements:** The customer and stakeholder needs that have been translated into a reason

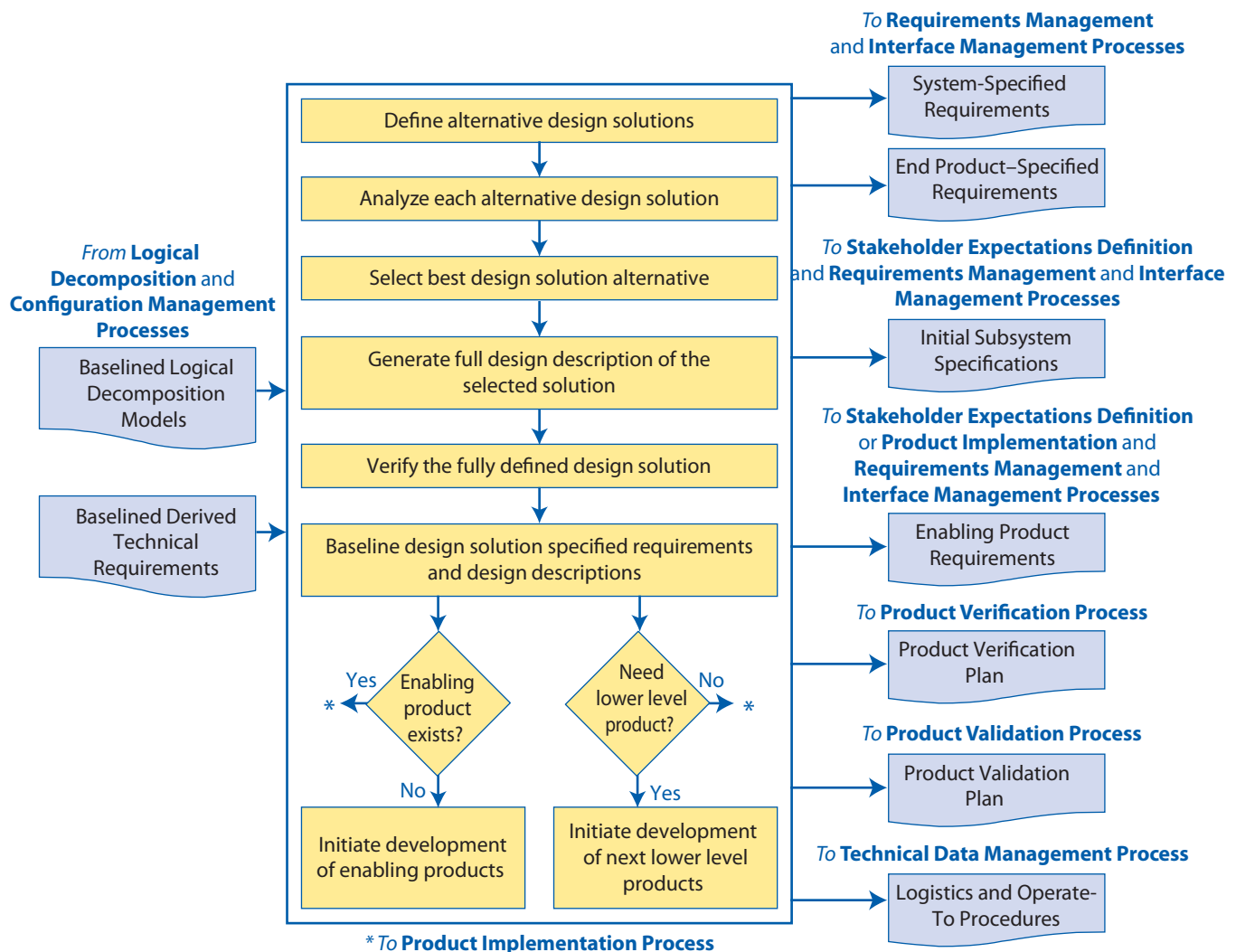


Figure 4.4-1 Design Solution Definition Process

4.0 System Design

ably complete set of validated requirements for the system, including all interface requirements.

- **Logical Decomposition Models:** Requirements decomposed by one or more different methods (e.g., function, time, behavior, data flow, states, modes, system architecture, etc.).

4.4.1.2 Process Activities

Define Alternative Design Solutions

The realization of a system over its life cycle involves a succession of decisions among alternative courses of action. If the alternatives are precisely defined and thoroughly understood to be well differentiated in the cost-effectiveness space, then the systems engineer can make choices among them with confidence.

To obtain assessments that are crisp enough to facilitate good decisions, it is often necessary to delve more deeply into the space of possible designs than has yet been done, as is illustrated in Figure 4.4-2. It should be realized, however, that this illustration represents neither the project life cycle, which encompasses the system development process from inception through disposal, nor the product development process by which the system design is developed and implemented.

Each create concepts step in Figure 4.4-2 involves a recursive and iterative design loop driven by the set of stake-

holder expectations where a strawman architecture/design, the associated ConOps, and the derived requirements are developed. These three products must be consistent with each other and will require iterations and design decisions to achieve this consistency. This recursive and iterative design loop is illustrated in Figure 4.0-1.

Each create concepts step also involves an assessment of potential capabilities offered by the continually changing state of technology and potential pitfalls captured through experience-based review of prior program/project lessons learned data. It is imperative that there be a continual interaction between the technology development process and the design process to ensure that the design reflects the realities of the available technology and that overreliance on immature technology is avoided. Additionally, the state of any technology that is considered enabling must be properly monitored, and care must be taken when assessing the impact of this technology on the concept performance. This interaction is facilitated through a periodic assessment of the design with respect to the maturity of the technology required to implement the design. (See Subsection 4.4.2.1 for a more detailed discussion of technology assessment.) These technology elements usually exist at a lower level in the PBS. Although the process of design concept development by the integration of lower level elements is a part of the systems engineering process, there is always a danger that the top-down process cannot keep up with the bottom-up process. Therefore, system architecture issues need to be resolved early so that the system can be modeled with sufficient realism to do reliable trade studies.

As the system is realized, its particulars become clearer—but also harder to change. The purpose of systems engineering is to make sure that the Design Solution Definition Process happens in a way that leads to the most cost-effective final system. The basic idea is that before those decisions that are hard to undo are made, the alternatives should be carefully assessed, particularly with respect to the maturity of the required technology.

Create Alternative Design Concepts

Once it is understood what the system is to accomplish, it is possible to devise a variety of ways that those goals can be met. Sometimes, that comes about as a consequence of considering alternative functional allocations and integrating available subsystem design options, all of which can have technologies at varying degrees of matu-

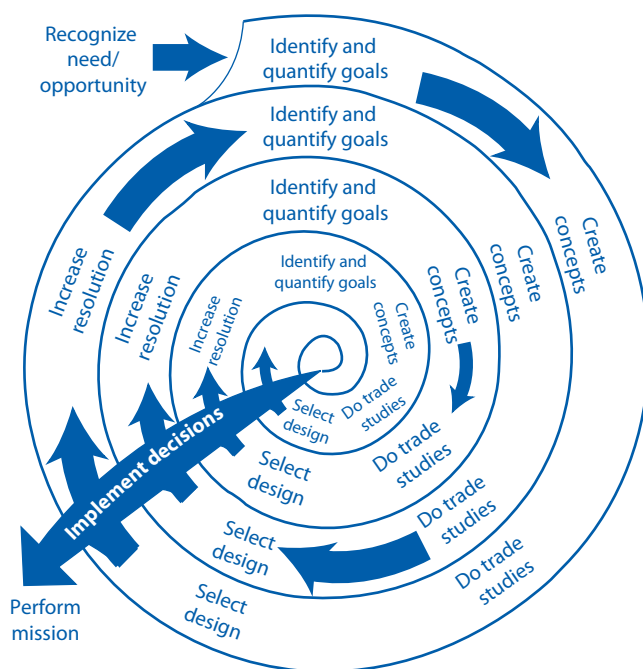


Figure 4.4-2 The doctrine of successive refinement

rity. Ideally, as wide a range of plausible alternatives as is consistent with the design organization's charter should be defined, keeping in mind the current stage in the process of successive refinement. When the bottom-up process is operating, a problem for the systems engineer is that the designers tend to become fond of the designs they create, so they lose their objectivity; the systems engineer often must stay an "outsider" so that there is more objectivity. This is particularly true in the assessment of the technological maturity of the subsystems and components required for implementation. There is a tendency on the part of technology developers and project management to overestimate the maturity and applicability of a technology that is required to implement a design. This is especially true of "heritage" equipment. The result is that critical aspects of systems engineering are often overlooked.

On the first turn of the successive refinement in Figure 4.4-2, the subject is often general approaches or strategies, sometimes architectural concepts. On the next, it is likely to be functional design, then detailed design, and so on. The reason for avoiding a premature focus on a single design is to permit discovery of the truly best design. Part of the systems engineer's job is to ensure that the design concepts to be compared take into account all interface requirements. "Did you include the cabling?" is a characteristic question. When possible, each design concept should be described in terms of controllable design parameters so that each represents as wide a class of designs as is reasonable. In doing so, the systems engineer should keep in mind that the potentials for change may include organizational structure, schedules, procedures, and any of the other things that make up a system. When possible, constraints should also be described by parameters.

Analyze Each Alternative Design Solution

The technical team analyzes how well each of the design alternatives meets the system goals (technology gaps, effectiveness, cost, schedule, and risk, both quantified and otherwise). This assessment is accomplished through the use of trade studies. The purpose of the trade study process is to ensure that the system architecture and design decisions move toward the best solution that can be achieved with the available resources. The basic steps in that process are:

- Devise some alternative means to meet the functional requirements. In the early phases of the project life-

cycle, this means focusing on system architectures; in later phases, emphasis is given to system designs.

- Evaluate these alternatives in terms of the MOEs and system cost. Mathematical models are useful in this step not only for forcing recognition of the relationships among the outcome variables, but also for helping to determine what the measures of performance must be quantitatively.
- Rank the alternatives according to appropriate selection criteria.
- Drop less promising alternatives and proceed to the next level of resolution, if needed.

The trade study process must be done openly and inclusively. While quantitative techniques and rules are used, subjectivity also plays a significant role. To make the process work effectively, participants must have open minds, and individuals with different skills—systems engineers, design engineers, specialty engineers, program analysts, decision scientists, and project managers—must cooperate. The right quantitative methods and selection criteria must be used. Trade study assumptions, models, and results must be documented as part of the project archives. The participants must remain focused on the functional requirements, including those for enabling products. For an in-depth discussion of the trade study process, see Section 6.8. The ability to perform these studies is enhanced by the development of system models that relate the design parameters to those assessments—but it does not depend upon them.

The technical team must consider a broad range of concepts when developing the system model. The model must define the roles of crew, hardware, and software in the system. It must identify the critical technologies required to implement the mission and must consider the entire life cycle, from fabrication to disposal. Evaluation criteria for selecting concepts must be established. Cost is always a limiting factor. However, other criteria, such as time to develop and certify a unit, risk, and reliability, also are critical. This stage cannot be accomplished without addressing the roles of operators and maintainers. These contribute significantly to life-cycle costs and to the system reliability. Reliability analysis should be performed based upon estimates of component failure rates for hardware. If probabilistic risk assessment models are applied, it may be necessary to include occurrence rates or probabilities for software faults or human error events. Assessments of the maturity of the required

4.0 System Design

technology must be done and a technology development plan developed.

Controlled modification and development of design concepts, together with such system models, often permits the use of formal optimization techniques to find regions of the design space that warrant further investigation.

Whether system models are used or not, the design concepts are developed, modified, reassessed, and compared against competing alternatives in a closed-loop process that seeks the best choices for further development. System and subsystem sizes are often determined during the trade studies. The end result is the determination of bounds on the relative cost-effectiveness of the design alternatives, measured in terms of the quantified system goals. (Only bounds, rather than final values, are possible because determination of the final details of the design is intentionally deferred.) Increasing detail associated with the continually improving resolution reduces the spread between upper and lower bounds as the process proceeds.

Select the Best Design Solution Alternative

The technical team selects the best design solution from among the alternative design concepts, taking into account subjective factors that the team was unable to quantify as well as estimates of how well the alternatives meet the quantitative requirements; the maturity of the available technology; and any effectiveness, cost, schedule, risk, or other constraints.

The Decision Analysis Process, as described in Section 6.8, should be used to make an evaluation of the alternative design concepts and to recommend the “best” design solution.

When it is possible, it is usually well worth the trouble to develop a mathematical expression, called an “objective function,” that expresses the values of combinations of possible outcomes as a single measure of cost-effectiveness, as illustrated in Figure 4.4-3, even if both cost and effectiveness must be described by more than one measure.

The objective function (or “cost function”) assigns a real number to candidate solutions or “feasible solutions” in the alternative space or “search space.” A feasible solution that minimizes (or maximizes, if that is the goal) the objective function is called an “optimal solution.” When

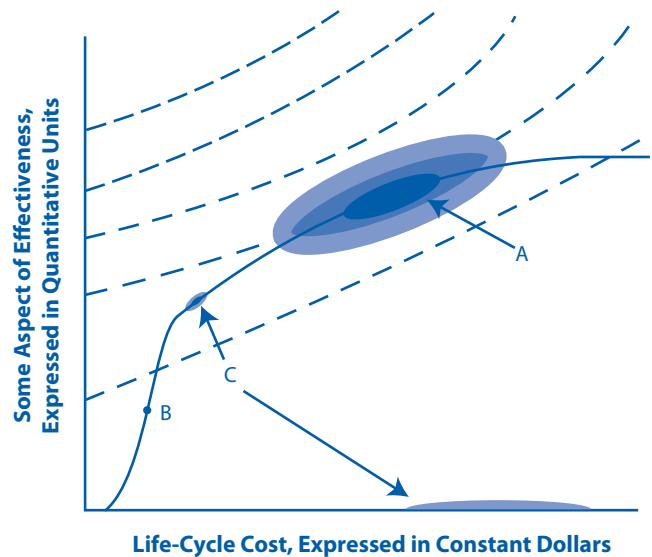


Figure 4.4-3 A quantitative objective function, dependent on life-cycle cost and all aspects of effectiveness

Note: The different shaded areas indicate different levels of uncertainty. Dashed lines represent constant values of objective function (cost-effectiveness). Higher values of cost-effectiveness are achieved by moving toward upper left. A, B, and C are design concepts with different risk patterns.

achievement of the goals can be quantitatively expressed by such an objective function, designs can be compared in terms of their value. Risks associated with design concepts can cause these evaluations to be somewhat nebulous (because they are uncertain and are best described by probability distributions).

In Figure 4.4-3, the risks are relatively high for design concept A. There is little risk in either effectiveness or cost for concept B, while the risk of an expensive failure is high for concept C, as is shown by the cloud of probability near the x axis with a high cost and essentially no effectiveness. Schedule factors may affect the effectiveness and cost values and the risk distributions.

The mission success criteria for systems differ significantly. In some cases, effectiveness goals may be much more important than all others. Other projects may demand low costs, have an immutable schedule, or require minimization of some kinds of risks. Rarely (if ever) is it possible to produce a combined quantitative measure that relates all of the important factors, even if it is expressed as a vector with several components. Even when that can be done, it is essential that the underlying factors and relationships be thoroughly revealed to and understood by the systems engineer. The systems engineer

must weigh the importance of the unquantifiable factors along with the quantitative data.

Technical reviews of the data and analyses, including technology maturity assessments, are an important part of the decision support packages prepared for the technical team. The decisions that are made are generally entered into the configuration management system as changes to (or elaborations of) the system baseline. The supporting trade studies are archived for future use. An essential feature of the systems engineering process is that trade studies are performed before decisions are made. They can then be baselined with much more confidence.

Increase the Resolution of the Design

The successive refinement process of Figure 4.4-2 illustrates a continuing refinement of the system design. At each level of decomposition, the baselined derived (and allocated) requirements become the set of high-level requirements for the decomposed elements, and the process begins again. One might ask, “When do we stop refining the design?” The answer is that the design effort precedes to a depth that is sufficient to meet several needs: the design must penetrate sufficiently to allow analytical validation of the design to the requirements; it must also have sufficient depth to support cost modeling and to convince a review team of a feasible design with performance, cost, and risk margins.

The systems engineering engine is applied again and again as the system is developed. As the system is realized, the issues addressed evolve and the particulars of the activity change. Most of the major system decisions (goals, architecture, acceptable life-cycle cost, etc.) are made during the early phases of the project, so the successive refinements do not correspond precisely to the phases of the system life cycle. Much of the system architecture can be seen even at the outset, so the successive refinements do not correspond exactly to development of the architectural hierarchy, either. Rather, they correspond to the successively greater resolution by which the system is defined.

It is reasonable to expect the system to be defined with better resolution as time passes. This tendency is formalized at some point (in Phase B) by defining a baseline system definition. Usually, the goals, objectives, and constraints are baselined as the requirements portion of the baseline. The entire baseline is then subjected to config-

uration control in an attempt to ensure that any subsequent changes are indeed justified and affordable.

At this point in the systems engineering process, there is a logical branch point. For those issues for which the process of successive refinement has proceeded far enough, the next step is to implement the decisions at that level of resolution. For those issues that are still insufficiently resolved, the next step is to refine the development further.

Fully Describe the Design Solution

Once the preferred design alternative has been selected and the proper level of refinement has been completed, then the design is fully defined into a final design solution that will satisfy the technical requirements. The design solution definition will be used to generate the end product specifications that will be used to produce the product and to conduct product verification. This process may be further refined depending on whether there are additional subsystems of the end product that need to be defined.

The scope and content of the full design description must be appropriate for the product life-cycle phase, the phase success criteria, and the product position in the PBS (system structure). Depending on these factors, the form of the design solution definition could be simply a simulation model or a paper study report. The technical data package evolves from phase to phase, starting with conceptual sketches or models and ending with complete drawings, parts list, and other details needed for product implementation or product integration. Typical output definitions from the Design Solution Definition Process are shown in Figure 4.4-1 and are described in Subsection 4.4.1.3.

Verify the Design Solution

Once an acceptable design solution has been selected from among the various alternative designs and documented in a technical data package, the design solution must next be verified against the system requirements and constraints. A method to achieve this verification is by means of a peer review to evaluate the resulting design solution definition. Guidelines for conducting a peer review are discussed in Section 6.7.

In addition, peer reviews play a significant role as a detailed technical component of higher level technical and

4.0 System Design

programmatic reviews. For example, the peer review of a component battery design can go into much more technical detail on the battery than the integrated power subsystem review. Peer reviews can cover the components of a subsystem down to the level appropriate for verifying the design against the requirements. Concerns raised at the peer review might have implications on the power subsystem design and verification and therefore must be reported at the next higher level review of the power subsystem.

The verification must show that the design solution definition:

- Is realizable within the constraints imposed on the technical effort;
- Has specified requirements that are stated in acceptable statements and have bidirectional traceability with the derived technical requirements, technical requirements, and stakeholder expectations; and
- Has decisions and assumptions made in forming the solution consistent with its set of derived technical requirements, separately allocated technical requirements, and identified system product and service constraints.

This design solution verification is in contrast to the verification of the end product described in the end product verification plan which is part of the technical data package. That verification occurs in a later life-cycle phase and is a result of the Product Verification Process (see Section 5.3) applied to the realization of the design solution as an end product.

Validate the Design Solution

The validation of the design solution is a recursive and iterative process as shown in Figure 4.0-1. Each alternative design concept is validated against the set of stakeholder expectations. The stakeholder expectations drive the iterative design loop in which a strawman architecture/design, the ConOps, and the derived requirements are developed. These three products must be consistent with each other and will require iterations and design decisions to achieve this consistency. Once consistency is achieved, functional analyses allow the study team to validate the design against the stakeholder expectations. A simplified validation asks the questions: Does the system work? Is the system safe and reliable? Is the system affordable? If the answer to any of these questions is no, then changes to the design or stakeholder expectations

will be required, and the process is started over again. This process continues until the system—architecture, ConOps, and requirements—meets the stakeholder expectations.

This design solution validation is in contrast to the validation of the end product described in the end product validation plan, which is part of the technical data package. That validation occurs in a later life-cycle phase and is a result of the Product Validation Process (see Section 5.4) applied to the realization of the design solution as an end product.

Identify Enabling Products

Enabling products are the life-cycle support products and services (e.g., production, test, deployment, training, maintenance, and disposal) that facilitate the progression and use of the operational end product through its life cycle. Since the end product and its enabling products are interdependent, they are viewed as a system. Project responsibility thus extends to responsibility for acquiring services from the relevant enabling products in each life-cycle phase. When a suitable enabling product does not already exist, the project that is responsible for the end product also can be responsible for creating and using the enabling product.

Therefore, an important activity in the Design Solution Definition Process is the identification of the enabling products that will be required during the life cycle of the selected design solution and then initiating the acquisition or development of those enabling products. Need dates for the enabling products must be realistically identified on the project schedules, incorporating appropriate schedule slack. Then firm commitments in the form of contracts, agreements, and/or operational plans must be put in place to ensure that the enabling products will be available when needed to support the product-line life-cycle phase activities. The enabling product requirements are documented as part of the technical data package for the Design Solution Definition Process.

An environmental test chamber would be an example of an enabling product whose use would be acquired at an appropriate time during the test phase of a space flight system.

Special test fixtures or special mechanical handling devices would be examples of enabling products that would have to be created by the project. Because of long

development times as well as oversubscribed facilities, it is important to identify enabling products and secure the commitments for them as early in the design phase as possible.

Baseline the Design Solution

As shown earlier in Figure 4.0-1, once the selected system design solution meets the stakeholder expectations, the study team baselines the products and prepares for the next life-cycle phase. Because of the recursive nature of successive refinement, intermediate levels of decomposition are often validated and baselined as part of the process. In the next level of decomposition, the baselined requirements become the set of high-level requirements for the decomposed elements, and the process begins again.

Baselining a particular design solution enables the technical team to focus on one design out of all the alternative design concepts. This is a critical point in the design process. It puts a stake in the ground and gets everyone on the design team focused on the same concept. When dealing with complex systems, it is difficult for team members to design their portion of the system if the system design is a moving target. The baselined design is documented and placed under configuration control. This includes the system requirements, specifications, and configuration descriptions.

While baselining a design is beneficial to the design process, there is a danger if it is exercised too early in the Design Solution Definition Process. The early exploration of alternative designs should be free and open to a wide range of ideas, concepts, and implementations. Baselining too early takes the inventive nature out of the concept exploration. Therefore baselining should be one of the last steps in the Design Solution Definition Process.

4.4.1.3 Outputs

Outputs of the Design Solution Definition Process are the specifications and plans that are passed on to the product realization processes. They contain the design-to, build-to, and code-to documentation that complies with the approved baseline for the system.

As mentioned earlier, the scope and content of the full design description must be appropriate for the product-line life-cycle phase, the phase success criteria, and the product position in the PBS.

Outputs of the Design Solution Definition Process include the following:

- **The System Specification:** The system specification contains the functional baseline for the system that is the result of the Design Solution Definition Process. The system design specification provides sufficient guidance, constraints, and system requirements for the design engineers to execute the design.
- **The System External Interface Specifications:** The system external interface specifications describe the functional baseline for the behavior and characteristics of all physical interfaces that the system has with the external world. These include all structural, thermal, electrical, and signal interfaces, as well as the human-system interfaces.
- **The End-Product Specifications:** The end-product specifications contain the detailed build-to and code-to requirements for the end product. They are detailed, exact statements of design particulars, such as statements prescribing materials, dimensions, and quality of work to build, install, or manufacture the end product.
- **The End-Product Interface Specifications:** The end-product interface specifications contain the detailed build-to and code-to requirements for the behavior and characteristics of all logical and physical interfaces that the end product has with external elements, including the human-system interfaces.
- **Initial Subsystem Specifications:** The end-product subsystem initial specifications provide detailed information on subsystems if they are required.
- **Enabling Product Requirements:** The requirements for associated supporting enabling products provide details of all enabling products. Enabling products are the life-cycle support products and services that facilitate the progression and use of the operational end product through its life cycle. They are viewed as part of the system since the end product and its enabling products are interdependent.
- **Product Verification Plan:** The end-product verification plan provides the content and depth of detail necessary to provide full visibility of all verification activities for the end product. Depending on the scope of the end product, the plan encompasses qualification, acceptance, prelaunch, operational, and disposal verification activities for flight hardware and software.

4.0 System Design

- **Product Validation Plan:** The end-product validation plan provides the content and depth of detail necessary to provide full visibility of all activities to validate the realized product against the baselined stakeholder expectations. The plan identifies the type of validation, the validation procedures, and the validation environment that are appropriate to confirm that the realized end product conforms to stakeholder expectations.
- **Logistics and Operate-to Procedures:** The applicable logistics and operate-to procedures for the system describe such things as handling, transportation, maintenance, long-term storage, and operational considerations for the particular design solution.

4.4.2 Design Solution Definition Guidance

4.4.2.1 Technology Assessment

As mentioned in the process description (Subsection 4.4.1), the creation of alternative design solutions involves assessment of potential capabilities offered by the continually changing state of technology. A continual interaction between the technology development process and the design process ensures that the design reflects the realities of the available technology. This interaction is facilitated through periodic assessment of the design with respect to the maturity of the technology required to implement the design.

After identifying the technology gaps existing in a given design concept, it will frequently be necessary to undertake technology development in order to ascertain viability. Given that resources will always be limited, it will be necessary to pursue only the most promising technologies that are required to enable a given concept.

If requirements are defined without fully understanding the resources required to accomplish needed technology developments then the program/project is at risk. Technology assessment must be done iteratively until requirements and available resources are aligned within an acceptable risk posture. Technology development plays a far greater role in the life cycle of a program/project than has been traditionally considered, and it is the role of the systems engineer to develop an understanding of the extent of program/project impacts—maximizing benefits and minimizing adverse effects. Traditionally, from a program/project perspective, technology development has been associated with the development and incor-

poration of any “new” technology necessary to meet requirements. However, a frequently overlooked area is that associated with the modification of “heritage” systems incorporated into different architectures and operating in different environments from the ones for which they were designed. If the required modifications and/or operating environments fall outside the realm of experience, then these too should be considered technology development.

To understand whether or not technology development is required—and to subsequently quantify the associated cost, schedule, and risk—it is necessary to systematically assess the maturity of each system, subsystem, or component in terms of the architecture and operational environment. It is then necessary to assess what is required in the way of development to advance the maturity to a point where it can successfully be incorporated within cost, schedule, and performance constraints. A process for accomplishing this assessment is described in Appendix G. Because technology development has the potential for such significant impacts on a program/project, technology assessment needs to play a role throughout the design and development process from concept development through Preliminary Design Review (PDR). Lessons learned from a technology development point of view should then be captured in the final phase of the program.

4.4.2.2 Integrating Engineering Specialties into the Systems Engineering Process

As part of the technical effort, specialty engineers in cooperation with systems engineering and subsystem designers often perform tasks that are common across disciplines. Foremost, they apply specialized analytical techniques to create information needed by the project manager and systems engineer. They also help define and write system requirements in their areas of expertise, and they review data packages, Engineering Change Requests (ECRs), test results, and documentation for major project reviews. The project manager and/or systems engineer needs to ensure that the information and products so generated add value to the project commensurate with their cost. The specialty engineering technical effort should be well integrated into the project. The roles and responsibilities of the specialty engineering disciplines should be summarized in the SEMP.

The specialty engineering disciplines included in this handbook are safety and reliability, Quality Assurance

(QA), ILS, maintainability, producibility, and human factors. An overview of these specialty engineering disciplines is provided to give systems engineers a brief introduction. It is not intended to be a handbook for any of these discipline specialties.

Safety and Reliability

Overview and Purpose

A reliable system ensures mission success by functioning properly over its intended life. It has a low and acceptable probability of failure, achieved through simplicity, proper design, and proper application of reliable parts and materials. In addition to long life, a reliable system is robust and fault tolerant, meaning it can tolerate failures and variations in its operating parameters and environments.

Safety and Reliability in the System Design Process

A focus on safety and reliability throughout the mission life cycle is essential for ensuring mission success. The fidelity to which safety and reliability are designed and built into the system depends on the information needed and the type of mission. For human-rated systems, safety and reliability is the primary objective throughout the design process. For science missions, safety and reliability should be commensurate with the funding and level of risk a program or project is willing to accept. Regardless of the type of mission, safety and reliability considerations must be an intricate part of the system design processes.

To realize the maximum benefit from reliability analysis, it is essential to integrate the risk and reliability analysts within the design teams. The importance of this cannot be overstated. In many cases, the reliability and risk analysts perform the analysis on the design after it has been formulated. In this case, safety and reliability features are added on or outsourced rather than designed in. This results in unrealistic analysis that is not focused on risk drivers and does not provide value to the design.

Risk and reliability analyses evolve to answer key questions about design trades as the design matures. Reliability analyses utilize information about the system, identify sources of risk and risk drivers, and provide an important input for decisionmaking. *NASA-STD-8729.1, Planning, Developing, and Maintaining an Effective Reliability and Maintainability (R&M) Program* outlines engineering activities that should be tailored

for each specific project. The concept is to choose an effective set of reliability and maintainability engineering activities to ensure that the systems designed, built, and deployed will operate successfully for the required mission life cycle.

In the early phases of a project, risk and reliability analyses help designers understand the interrelationships of requirements, constraints, and resources, and uncover key relationships and drivers so they can be properly considered. The analyst must help designers go beyond the requirements to understand implicit dependencies that emerge as the design concept matures. It is unrealistic to assume that design requirements will correctly capture all risk and reliability issues and “force” a reliable design. The systems engineer should develop a system strategy mapped to the PBS on how to allocate and coordinate reliability, fault tolerance, and recovery between systems both horizontally and vertically within the architecture to meet the total mission requirements. System impacts of designs must play a key role in the design. Making designers aware of impacts of their decisions on overall mission reliability is key.

As the design matures, preliminary reliability analysis occurs using established techniques. The design and concept of operations should be thoroughly examined for accident initiators and hazards that could lead to mishaps. Conservative estimates of likelihood and consequences of the hazards can be used as a basis for applying design resources to reduce the risk of failures. The team should also ensure that the goals can be met and failure modes are considered and take into account the entire system.

During the latter phases of a project, the team uses risk assessments and reliability techniques to verify that the design is meeting its risk and reliability goals and to help develop mitigation strategies when the goals are not met or discrepancies/failures occur.

Analysis Techniques and Methods

This subsection provides a brief summary of the types of analysis techniques and methods.

- Event sequence diagrams/event trees are models that describe the sequence of events and responses to off-nominal conditions that can occur during a mission.
- Failure Modes and Effects Analyses (FMEAs) are bottom-up analyses that identify the types of failures

4.0 System Design

that can occur within a system and identify the causes, effects, and mitigating strategies that can be employed to control the effects of the failures.

- Qualitative top-down logic models identify how failures within a system can combine to cause an undesired event.
- Quantitative logic models (probabilistic risk assessment) extend the qualitative models to include the likelihood of failure. These models involve developing failure criteria based on system physics and system success criteria, and employing statistical techniques to estimate the likelihood of failure along with uncertainty.
- Reliability block diagrams are diagrams of the elements to evaluate the reliability of a system to provide a function.
- Preliminary Hazard Analysis (PHA) is performed early based on the functions performed during the mission. Preliminary hazard analysis is a “what if” process that considers the potential hazard, initiating event scenarios, effects, and potential corrective measures and controls. The objective is to determine if the hazard can be eliminated, and if not, how it can be controlled.
- Hazard analysis evaluates the completed design. Hazard analysis is a “what if” process that considers the potential hazard, initiating event, effects, and potential corrective measures and controls. The objective is to determine if the hazard can be eliminated, and if not, how it can be controlled.
- Human reliability analysis is a method to understand how human failures can lead to system failure and estimate the likelihood of those failures.
- Probabilistic structural analysis provides a way to combine uncertainties in materials and loads to evaluate the failure of a structural element.
- Sparing/logistics models provide a means to estimate the interactions of systems in time. These models include ground-processing simulations and mission campaign simulations.

Limitations on Reliability Analysis

The engineering design team must understand that reliability is expressed as the probability of mission success. Probability is a mathematical measure expressing the likelihood of occurrence of a specific event. Therefore, probability estimates should be based on engineering

and historical data, and any stated probabilities should include some measure of the uncertainty surrounding that estimate.

Uncertainty expresses the degree of belief analysts have in their estimates. Uncertainty decreases as the quality of data and understanding of the system improve. The initial estimates of failure rates or failure probability might be based on comparison to similar equipment, historical data (heritage), failure rate data from handbooks, or expert elicitation.

In summary,

- Reliability estimates express probability of success.
- Uncertainty should be included with reliability estimates.
- Reliability estimates combined with FMEAs provide additional and valuable information to aid in the decisionmaking process.

Quality Assurance

Even with the best designs, hardware fabrication and testing are subject to human error. The systems engineer needs to have some confidence that the system actually produced and delivered is in accordance with its functional, performance, and design requirements. QA provides an independent assessment to the project manager/systems engineer of the items produced and processes used during the project life cycle. The project manager/systems engineer must work with the quality assurance engineer to develop a quality assurance program (the extent, responsibility, and timing of QA activities) tailored to the project it supports.

QA is the mainstay of quality as practiced at NASA. *NPD 8730.5, NASA Quality Assurance Program Policy* states that NASA’s policy is “to comply with prescribed requirements for performance of work and to provide for independent assurance of compliance through implementation of a quality assurance program.” The quality function of Safety and Mission Assurance (SMA) ensures that both contractors and other NASA functions do what they say they will do and say what they intend to do. This ensures that end product and program quality, reliability, and overall risk are at the level planned.

The Systems Engineer’s Relationship to QA

As with reliability, producibility, and other characteristics, quality must be designed as an integral part of any

system. It is important that the systems engineer understands SMA's safeguarding role in the broad context of total risk and supports the quality role explicitly and vigorously. All of this is easier if the SMA quality function is actively included and if quality is designed in with buy-in by all roles, starting at concept development. This will help mitigate conflicts between design and quality requirements, which can take on the effect of "tolerance stacking."

Quality is a vital part of risk management. Errors, variability, omissions, and other problems cost time, program resources, taxpayer dollars, and even lives. It is incumbent on the systems engineer to know how quality affects their projects and to encourage best practices to achieve the quality level.

Rigid adherence to procedural requirements is necessary in high-risk, low-volume manufacturing. In the absence of large samples and long production runs, compliance to these written procedures is a strong step toward ensuring process, and, thereby, product consistency. To address this, NASA requires QA programs to be designed to mitigate risks associated with noncompliance to those requirements.

There will be a large number of requirements and procedures thus created. These must be flowed down to the supply chain, even to lowest tier suppliers. For circumstances where noncompliance can result in loss of life or loss of mission, there is a requirement to insert into procedures Government Mandatory Inspection Points (GMIPs) to ensure 100 percent compliance with safety/mission-critical attributes. Safety/mission-critical attributes include hardware characteristics, manufacturing process requirements, operating conditions, and functional performance criteria that, if not met, can result in loss of life or loss of mission. There will be in place a Program/Project Quality Assurance Surveillance Plan (PQASP) as mandated by Federal Acquisition Regulation (FAR) Subpart 46.4. Preparation and content for PQASPs are outlined in *NPR 8735.2, Management of Government Quality Assurance Functions for NASA Contracts*. This document covers quality assurance requirements for both low-risk and high-risk acquisitions and includes functions such as document review, product examination, process witnessing, quality system evaluation, nonconformance reporting and corrective action, planning for quality assurance and surveillance, and GMIPs. In addition, most NASA projects are required to

adhere to either ISO 9001 (noncritical work) or AS9100 (critical work) requirements for management of quality systems. Training in these systems is mandatory for most NASA functions, so knowledge of their applicability by the systems engineer is assumed. Their texts and intent are strongly reflected in NASA's quality procedural documents.

Integrated Logistics Support

The objective of ILS activities within the systems engineering process is to ensure that the product system is supported during development (Phase D) and operations (Phase E) in a cost-effective manner. ILS is particularly important to projects that are reusable or serviceable. Projects whose primary product does not evolve over its operations phase typically only apply ILS to parts of the project (for example, the ground system) or to some of the elements (for example, transportation). ILS is primarily accomplished by early, concurrent consideration of supportability characteristics; performing trade studies on alternative system and ILS concepts; quantifying resource requirements for each ILS element using best practices; and acquiring the support items associated with each ILS element. During operations, ILS activities support the system while seeking improvements in cost-effectiveness by conducting analyses in response to actual operational conditions. These analyses continually reshape the ILS system and its resource requirements. Neglecting ILS or poor ILS decisions invariably have adverse effects on the life-cycle cost of the resultant system. Table 4.4-1 summarizes the ILS disciplines.

ILS planning should begin early in the project life cycle and should be documented. This plan should address the elements above including how they will be considered, conducted, and integrated into the systems engineering process needs.

Maintainability

Maintainability is defined as the measure of the ability of an item to be retained in or restored to specified conditions when maintenance is performed by personnel having specified skill levels, using prescribed procedures and resources, at each prescribed level of maintenance. It is the inherent characteristics of a design or installation that contribute to the ease, economy, safety, and accuracy with which maintenance actions can be performed.

Table 4.4-1 ILS Technical Disciplines

Technical Discipline	Definition
Maintenance support planning	Ongoing and iterative planning, organization, and management activities necessary to ensure that the logistics requirements for any given program are properly coordinated and implemented
Design interface	The interaction and relationship of logistics with the systems engineering process to ensure that supportability influences the definition and design of the system so as to reduce life-cycle cost
Technical data and technical publications	The recorded scientific, engineering, technical, and cost information used to define, produce, test, evaluate, modify, deliver, support, and operate the system
Training and training support	Encompasses all personnel, equipment, facilities, data/documentation, and associated resources necessary for the training of operational and maintenance personnel
Supply support	Actions required to provide all the necessary material to ensure the system's supportability and usability objectives are met
Test and support equipment	All tools, condition-monitoring equipment, diagnostic and checkout equipment, special test equipment, metrology and calibration equipment, maintenance fixtures and stands, and special handling equipment required to support operational maintenance functions
Packaging, handling, storage, and transportation	All materials, equipment, special provisions, containers (reusable and disposable), and supplies necessary to support the packaging, safety and preservation, storage, handling, and transportation of the prime mission-related elements of the system, including personnel, spare and repair parts, test and support equipment, technical data computer resources, and mobile facilities
Personnel	Involves identification and acquisition of personnel with skills and grades required to operate and maintain a system over its lifetime
Logistics facilities	All special facilities that are unique and are required to support logistics activities, including storage buildings and warehouses and maintenance facilities at all levels
Computer resources support	All computers, associated software, connecting components, networks, and interfaces necessary to support the day-to-day flow of information for all logistics functions

Source: Blanchard, *System Engineering Management*.

Role of the Maintainability Engineer

Maintainability engineering is another major specialty discipline that contributes to the goal of a supportable system. This is primarily accomplished in the systems engineering process through an active role in implementing specific design features to facilitate safe and effective maintenance actions in the predicted physical environments, and through a central role in developing the ILS system. Example tasks of the maintainability engineer include: developing and maintaining a system maintenance concept, establishing and allocating maintainability requirements, performing analysis to quantify the system's maintenance resource requirements, and verifying the system's maintainability requirements.

Producibility

Producibility is a system characteristic associated with the ease and economy with which a completed design can be transformed (i.e., fabricated, manufactured, or coded) into a hardware and/or software realization. While major NASA systems tend to be produced in small quantities, a particular producibility feature can be crit-

ical to a system's cost-effectiveness, as experience with the shuttle's thermal tiles has shown. Factors that influence the producibility of a design include the choice of materials, simplicity of design, flexibility in production alternatives, tight tolerance requirements, and clarity and simplicity of the technical data package.

Role of the Production Engineer

The production engineer supports the systems engineering process (as a part of the multidisciplinary product development team) by taking an active role in implementing specific design features to enhance producibility and by performing the production engineering analyses needed by the project. These tasks and analyses include:

- Performing the manufacturing/fabrication portion of the system risk management program. This is accomplished by conducting a rigorous production risk assessment and by planning effective risk mitigation actions.
- Identifying system design features that enhance producibility. Efforts usually focus on design simplifica-

tion, fabrication tolerances, and avoidance of hazardous materials.

- Conducting producibility trade studies to determine the most cost-effective fabrication/manufacturing process.
- Assessing production feasibility within project constraints. This may include assessing contractor and principal subcontractor production experience and capability, new fabrication technology, special tooling, and production personnel training requirements.
- Identifying long-lead items and critical materials.
- Estimating production costs as a part of life-cycle cost management.
- Supporting technology readiness assessments.
- Developing production schedules.
- Developing approaches and plans to validate fabrication/manufacturing processes.

The results of these tasks and production engineering analyses are documented in the manufacturing plan with a level of detail appropriate to the phase of the project. The production engineer also participates in and contributes to major project reviews (primarily PDR and Critical Design Review (CDR)) on the above items, and to special interim reviews such as the PRR.

Prototypes

Experience has shown that prototype systems can be effective in enabling efficient producibility even when building only a single flight system. Prototypes are built early in the life cycle and they are made as close to the flight item in form, fit, and function as is feasible at that stage of the development. The prototype is used to “wring out” the design solution so that experience gained from the prototype can be fed back into design changes that will improve the manufacture, integration, and maintainability of a single flight item or the production run of several flight items. Unfortunately, prototypes are often deleted from projects to save cost. Along with that decision, the project accepts an increased risk in the development phase of the life cycle. Fortunately, advancements in computer-aided design and manufacturing have mitigated that risk somewhat by enabling the designer to visualize the design and “walk through” the integration sequence to uncover problems before they become a costly reality.

Human Factors Engineering

Overview and Purpose

Consideration of human operators and maintainers of systems is a critical part of the design process. Human factors engineering is the discipline that studies the human-system interfaces and provides requirements, standards, and guidelines to ensure the human component of the integrated system is able to function as intended. Human roles include operators (flight crews and ground crews), designers, manufacturers, ground support, maintainers, and passengers. Flight crew functions include system operation, troubleshooting, and in-flight maintenance. Ground crew functions include spacecraft and ground system manufacturing, assembly, test, checkout, logistics, ground maintenance, repair, refurbishment, launch control, and mission control.

Human factors are generally considered in four categories. The first is anthropometry and biomechanics—the physical size, shape, and strength of the humans. The second is sensation and perception—primarily vision and hearing, but senses such as touch are also important. The environment is a third factor—ambient noise and lighting, vibration, temperature and humidity, atmospheric composition, and contaminants. Psychological factors comprise memory; information processing components such as pattern recognition, decisionmaking, and signal detection; and affective factors—e.g., emotions, cultural patterns, and habits.

Human Factors Engineering in the System Design Process

- **Stakeholder Expectations:** The operators, maintainers, and passengers are all stakeholders in the system. The human factors specialist identifies roles and responsibilities that can be performed by humans and scenarios that exceed human capabilities. The human factors specialist ensures that system operational concept development includes task analysis and human/system function allocation. As these are refined, function allocation distributes operator roles and responsibilities for subtasks to the crew, external support teams, and automation. (For example, in aviation, tasks may be allocated to crew, air traffic controllers, or autopilots. In spacecraft, tasks may be performed by crew, mission control, or onboard systems.)

4.0 System Design

- **Requirements Definition:** Human factors requirements for spacecraft and space habitats are program/project dependent, derived from *NASA-STD-3001, NASA Space Flight Human System Standard Volume 1: Crew Health*. Other human factors requirements of other missions and Earth-based activities for human space flight missions are derived from human factors standards such as *MIL-STD-1472, Human Engineering*; *NUREG-0700, Human-System Interface Design Review Guidelines*; and the Federal Aviation Administration's *Human Factors Design Standard*.
- **Technical Solution:** Consider the human as a central component when doing logical decomposition and developing design concepts. The users—operators or maintainers—will not see the entire system as the designer does, only as the system interfaces with them. In engineering design reviews, human factors specialists promote the usability of the design solution. With early involvement, human factors assessments may catch usability problems at very early stages. For example, in one International Space Station payload design project, a human factors assessment of a very early block diagram of the layout of stowage and hardware identified problems that would have made operations very difficult. Changes were made to the conceptual design at negligible cost—i.e., rearranging conceptual block diagrams based on the sequence in which users would access items.
- **Usability Evaluations of Design Concepts:** Evaluations can be performed easily using rapid prototyping tools for hardware and software interfaces, standard human factors engineering data-gathering and analysis tools, and metrics such as task completion time and number of errors. Systematically collected subjective reports from operators also provide useful data. New technologies provide detailed objective information—e.g., eye tracking for display and control layout assessment. Human factors specialists provide assessment capabilities throughout the iterative design process.
- **Verification:** As mentioned, verification of requirements for usability, error rates, task completion times, and workload is challenging. Methods range from tests with trained personnel in mockups and simulators, to models of human performance, to inspection by experts. As members of the systems engineering team, human factors specialists provide verification guidance from the time requirements are first developed.

Human Factors Engineering Analyses Techniques and Methods

Example methods used to provide human performance data, predict human-system performance, and evaluate human-system designs include:

- **Task Analysis:** Produces a detailed description of the things a person must do in a system to accomplish a task, with emphasis on requirements for information presentation, decisions to be made, task times, operator actions, and environmental conditions.
- **Timeline Analysis:** Follows from task analysis. Durations of tasks are identified in task analyses, and the times at which these tasks occur are plotted in graphs, which also show the task sequences. The purpose is to identify requirements for simultaneous incompatible activities and activities that take longer than is available. Timelines for a given task can describe the activities of multiple operators or crewmembers.
- **Modeling and Simulation:** Models or mockups to make predictions about system performance, compare configurations, evaluate procedures, and evaluate alternatives. Simulations can be as simple as positioning a graphical human model with realistic anthropometric dimensions with a graphical model of an operator station, or they can be complex stochastic models capturing decision points, error opportunities, etc.
- **Usability Testing:** Based on a task analysis and preliminary design, realistic tasks are carried out in a controlled environment with monitoring and recording equipment. Objective measures such as performance time and number of errors are evaluated; subjective ratings are collected. The outputs systematically report on strengths and weaknesses of candidate design solutions.
- **Workload Assessment:** Measurement on a standardized scale such as the NASA-TLX or the Cooper-Harper rating scales of the amount and type of work. It assesses operator and crew task loading, which determines the ability of a human to perform the required tasks in the desired time with the desired accuracy.
- **Human Error and Human Reliability Assessment:** Top-down (fault tree analyses) and bottom-up (human factors process failure modes and effects analysis) analyses. The goal is to promote human reliability by creating a system that can tolerate and recover from human errors. Such a system must also support the human role in adding reliability to the system.

Roles of the Human Factors Specialist

The human factors specialist supports the systems engineering process by representing the users' and maintainers' requirements and capabilities throughout the design, production, and operations stages. Human factors specialists' roles include:

- Identify applicable requirements based on Agency standards for human-system integration during the requirements definition phase.
- Support development of mission concepts by providing information on human performance capabilities and limitations.
- Support task analysis and function allocation with information on human capabilities and limitations.
- Identify system design features that enhance usability. This integrates knowledge of human performance capabilities and design features.
- Support trade studies by providing data on effects of alternative designs on time to complete tasks, workload, and error rates.
- Support trade studies by providing data on effects of alternative designs on skills and training required to operate the system.
- Support design reviews to ensure compliance with human-systems integration requirements.
- Conduct evaluations using mockups and prototypes to provide detailed data on user performance.
- Support development of training and maintenance procedures in conjunction with hardware designers and mission planners.
- Collect data on human-system integration issues during operations to inform future designs.